Learning Summary Report

Name: Yihao Yu

Student ID: 35567082

Date: 20 October 2026

Self-Assessment

Grade demonstrated: 4K

Declaration

I declare that this portfolio is my individual work. I have not copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part of this submission been written for me by another person. The extent to which I have used generative AI is described and appropriately attributed.

Signature: Yihao Yu

Statement of Achievements

I am applying for a 4K grade in FIT2085. My portfolio demonstrates progress from basic C++ programming and sorting to modular solutions using linked lists, stacks, queues, trees, heaps and hash tables. The P, C, D and HD tasks provide evidence of implementation, complexity analysis and correctness reasoning.

My completed 4K tasks are Faulty Check-in and Perfect Test. The reports in my 4K portfolio each contain an AC verdict and a score of 100/100, together with handwritten analysis and a new problem statement. Their folders also contain C++ source and video recordings. Faulty Check-in uses a queue and a stack in O(n) total time; Perfect Test uses a min heap in O(n log n) time. These tasks support my ability to select suitable strategies, justify their behaviour and transfer the ideas to new contexts.

Achievement of Unit Learning Outcomes

Learning outcome 1 Analyse computational problems

P2 compares selection, insertion and bubble sort through the costs of array reads and writes. I learned that algorithms with the same quadratic worst-case bound may have different practical operation costs. P4 analyses a larger engagement-score program by combining sorting and searching costs. P6 shows why splitting into more sublists does not automatically improve performance when merging is sequential. In Perfect Test, recognising that every retained prefix must remain non-negative led me to process values in their original order and use a min heap to support removal decisions.

Learning outcome 2 Implement use and test data structures

My C++ work implements and uses linked structures, sorting algorithms, heaps, trees and hash tables. P6 records tests for sorted input, reverse order, duplicates and negative values. C1 applies templates to reusable implementations and comparisons for custom types. The accepted 100/100 results for both 4K tasks add external test evidence. I understand that acceptance covers the judge tests, while readability, maintainability and broader correctness still require code review and clear reasoning.

Learning outcome 3 Design modular algorithmic solutions

C2 combines a generic linked list of batches with a binary search tree for each batch. Sorting prepares each batch for tree construction, and threshold searches produce candidates that are compared across batches. P8 combines a max heap for trending posts with a search tree for usernames. Faulty Check-in separates FIFO queue operations from LIFO stack operations. These designs show how I assign clear responsibilities to abstract data types and combine them to meet a larger requirement.

Achievement of Unit Learning Outcomes continued

Learning outcome 4 Use relevant tools and technologies

P1 records the clang++ compilation command and how to run the executable. My Git history records additions, refactoring and improvements to comments, including work on the 4K solutions. The 4K reports organise handwritten notes, judge verdicts and video links as evidence that can be inspected alongside the source. These practices help me reproduce work and communicate changes. When using AI assistance, I need to check suggestions against the implementation and remain responsible for understanding the algorithm.

Learning outcome 5 Plan monitor and reflect on development

P0 records my 4K target and why data structures and algorithms matter to my future studies. Weekly folders, task reports and repository revisions provide a record of development. P10 reflects on applied and Buddy Group activities. My Perfect Test notes, reconstructed from my recollection of the first attempt, show a specific revision: I corrected a misunderstanding of the requirement and changed the heap strategy. I can improve my monitoring by documenting the reason for each change and the tests used to check it.

Learning outcome 6 Communicate technical reasoning

P7 explains div_3 through initialisation, maintenance and termination of a loop invariant. C2 explains the branches in its threshold search. HD1 counts lifetime pushes and pops to justify linear total stack work, while HD2 distinguishes expected logarithmic tree height from a possible linear worst case. My 4K reports add worked traces and the new problems Single-Lane Cargo Station and A Day Full of Activities. These examples show how I communicate correctness, performance assumptions and implementation choices through prose, diagrams and code.

Reflections

Most important thing learnt

The most important thing I learnt is to translate the problem constraints into a property the algorithm must preserve. Perfect Test made this concrete. My initial approach focused on the final total and used a max heap, but the score must stay non-negative at every point and retained questions must keep their order. Processing scores sequentially and removing the lowest selected score when necessary addresses those constraints. This experience changed my approach: I now try to state the required invariant before deciding how to implement a solution.

Most interesting topic or task

Perfect Test was especially interesting because a local removal decision supports a larger optimisation goal. A min heap identifies the lowest selected score, and removing it gives the greatest increase in the current total among single-removal choices. The heap manages the selected scores without determining their chronological order. Developing A Day Full of Activities helped me express the same idea in terms of energy gained and consumed. I gained a clearer understanding of how an algorithmic structure can remain useful when the surrounding story changes.

Challenges Overcome

One challenge was distinguishing what a structure stores from the order in which the problem must be processed. My reconstructed Perfect Test notes show how rereading the statement and tracing a small example helped me replace the initial max-heap approach with a min heap. Faulty Check-in presented a related modelling challenge: the second line is described as a queue, but adding and removing at its front gives LIFO behaviour. Writing down the permitted operations made the stack model clear. These tasks taught me to work from operational rules and use traces to test an interpretation before coding.

What next?

I want to learn more about balanced and randomised search trees, automated invariant checks and memory-safe C++ design. HD2 gives me a starting point for understanding rotations and the difference between expected and worst-case performance. My linked structures also make ownership and cleanup important. I want to build small automated checks for tree ordering, parent pointers and heap properties, so that later revisions can be checked more reliably than by inspecting printed output alone.

Reflections continued

My initial learning plan

My engagement contract in P0 set a 4K target and connected the unit to future study and software development. The weekly notes and progression through P, C, D, HD and 4K work gave my learning a practical structure. Completing the two 4K tasks and explaining their strategies represents progress towards that target. However, the timeline field in my engagement contract was left blank, so I cannot assess progress against a detailed original schedule. Next time, I would set milestones for implementation, testing and reflection, and record progress against them each week.

Retrospective Advice for Myself

I would tell myself to read every constraint carefully, trace one small example and write the invariant before choosing a data structure. This would have helped me identify the prefix requirement in Perfect Test earlier. I would also analyse the actual implementation rather than rely on the familiar name of an algorithm. P6 shows why the merge strategy matters, and HD1 shows why nested loops can still have linear total work. I would keep expected outputs with my test inputs and record why each revision was made.

Monash

Fundamentals of Algorithms

Yihao Yu

Portfolio Submission

Submitted By:
Yihao Yu
yyuu0140

Tutor:
Ali Toosi

20 Oct 2026
02:53 PM Australia/Melbourne



Produced by OnTrack

Contents

1 Learning Summary Report **1**

2 Overall Task Status **2**

3 Learning Outcomes **3**
3.1 ULO1 Problem Solving . . . 3
3.2 ULO2 Programming . . . 3
3.3 ULO3 Design . . . 3
3.4 ULO4 Digital Mindset . . . 3
3.5 ULO5 Learning & Development . . . 3
3.6 ULO6 Communication . . . 3

4 Engagement Contract **4**

5 C++ Starter Pack **7**

6 Sorting on Hardware **17**

7 Pointing Linked Arrays **27**

8 Complex Complexities **36**

9 Stacks & Queues **44**

10 Templates **56**

11 Custom Merge Sort **65**

12 Invariant Proofs **75**

13 Batched Queries **80**

14 Supercharged BSTs **100**

15 Tree Structures **113**

16 Monotonic Stack **124**

17 List of Followers **135**

18 Infinite Hash Table **146**

19 BST and Heap **160**

20 Applied Engagement **179**

21 Problem Solving Workbook **183**

22 HD Interview **186**

2 Overall Task Status

Task	Status	Times Assessed
P0 Engagement Contract	Complete	1
P1 C++ Starter Pack	Complete	1
P10 Applied Engagement	Complete	1
P2 Sorting on Hardware	Complete	2
P3 Pointing Linked Arrays	Complete	2
P4 Complex Complexities	Complete	2
P5 Stacks & Queues	Complete	1
P6 Custom Merge Sort	Complete	1
C1 Templates	Complete	1
P7 Invariant Proofs	Complete	1
P8 Tree Structures	Complete	1
C2 Batched Queries	Complete	2
D1 Supercharged BSTs	Complete	1
HD1 Monotonic Stack	Complete	4
P9 List of Followers	Complete	1
HD2 BST and Heap	Complete	1
D2 Infinite Hash Table	Complete	1
4K Problem Solving Workbook	Ready for Feedback	0
HD3 HD Interview	Ready for Feedback	0

3 Learning Outcomes

3.1 ULO1 Problem Solving

Analyse computational problems to identify suitable algorithmic strategies, data representations and performance considerations.

3.2 ULO2 Programming

Demonstrate understanding of data structures and algorithms by implementing, using, and testing them in ways that support correctness, readability and maintainability.

3.3 ULO3 Design

Design modular algorithmic solutions using appropriate abstract data types, including lists, stacks, queues, trees, heaps and hash tables.

3.4 ULO4 Digital Mindset

Demonstrate awareness and working knowledge of relevant tools and technologies, and use them effectively to increase productivity and improve quality, such as IDEs, AI, and Version Control Systems.

3.5 ULO5 Learning & Development

Plan, monitor, and reflect on the development of your algorithmic thinking and implementation practice through focused problem solving, feedback and iterative improvement.

3.6 ULO6 Communication

Communicate algorithmic reasoning and correctness, implementation choices, performance trade-offs and testing evidence using appropriate technical terminology and representations.

4 Engagement Contract

What the teaching team provides, what you are expected to do.

Outcome	Description
TLO1	Understand how the unit is structured, what is provided to you and what's expected of you, and how you can get the most out of the unit.

Date	Author	Comment
2026/07/27 22:37	Yihao Yu	Working On It
2026/07/31 23:01	Yihao Yu	Planned date adjusted: 27 Jul - 02 Aug.
2026/08/02 23:41	Yihao Yu	Planned date adjusted: 27 Jul - 03 Aug.
2026/08/03 10:21	Yihao Yu	Ready for Feedback
2026/08/03 23:23	Gary Zheng	Hi Yihao, welcome to FIT1008. I will be your tutor for this semester. Please come to class and have a discussion with me so we can get this task ticked off. See you soon!
2026/08/03 23:23	Gary Zheng	Discuss
2026/08/04 12:23	Yihao Yu	Thank you so much! I really appreciate it!
2026/08/04 14:01	Gary Zheng	Complete
2026/08/04 14:01	Gary Zheng	Discussed in class

Monash

Fundamentals of Algorithms

OnTrack Submission

Engagement Contract

Submitted By:
Yihao Yu
yyuu0140
2026/08/03 10:21

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Understand how the unit is structured, what is provided to you and what's expected of you, and how you can get the most out of the unit.	

August 3, 2026



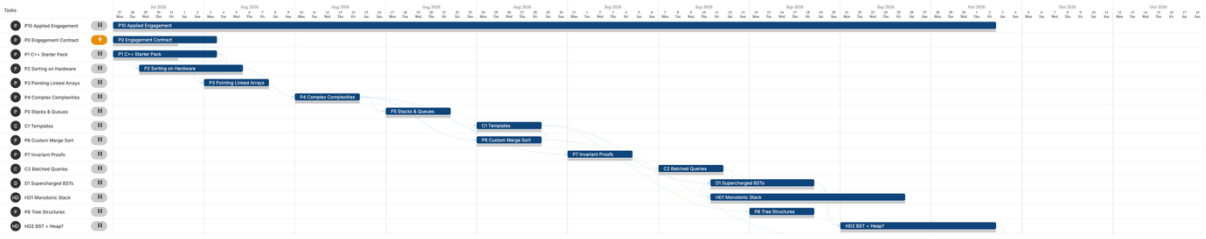
Engagement Contract

- Full Name: Yihao Yu
- Student ID: 35567082

How does learning about Data Structures and Algorithms support you in your future career and the rest of your studies?
Learning about Data Structures and Algorithms is essential because it provides me with the foundation of writing efficiency and maintainable software. From an academic perspective, Data structure helps students how to organise and process data efficiently. Many advanced subjects, such as operating systems, databases, artificial intelligence rely on concepts like trees, graphs, and so on. For future career, data structure and algorithm is widely used in software development. Choosing the right data structure would improve software's performance significantly. Data structures and algorithms is of great importance in the interviews because it could demonstrate programming abilities and logical thinking.
Overall, learning Data structures and algorithms provides the theorical foundation and practical skills needed no matter in later university courses or in my future carer life.

What grade are you aiming for?
4K

What is your planned timeline of completing the tasks?



Have you read the provided engagement contract on this task, and reading material of Week 1, and understood how this unit operates, including the assessment regime and how the final marks will be calculated?
Yes, I have

Signature & Date
02/08/2026



5 C++ Starter Pack

Getting started with C++ as the main language of the unit.

Outcome	Description
TLO1	Learn about C++ and understand how to set up your computer for developing C++ programs.

Date	Author	Comment
2026/07/31 23:01	Yihao Yu	Planned date adjusted: 27 Jul - 02 Aug.
2026/08/02 23:41	Yihao Yu	Planned date adjusted: 27 Jul - 03 Aug.
2026/08/03 14:28	Yihao Yu	I am not sure if the format of my report is accurate or not. Thanks for your apply in advance! I really appreciate it!
2026/08/03 14:28	Yihao Yu	Ready for Feedback
2026/08/03 14:28	Gary Zheng	Tests complete: 1 / 1
2026/08/03 23:34	Gary Zheng	Hi Yihao, looks all good on my end. Code is correct, good work. Just one optimisation you can do, once is_sorted is set to false, you can return it immediately rather than going through the rest of the array.Please have a discussion with me in class so we can tick this task off.
2026/08/03 23:34	Gary Zheng	Discuss
2026/08/04 12:24	Yihao Yu	Thank you so much! I really appreciate it!
2026/08/04 14:00	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

C++ Starter Pack

Submitted By:
Yihao Yu
yyuu0140
2026/08/03 14:28

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about C++ and understand how to set up your computer for developing C++ programs.	

August 3, 2026



C++ Cheat sheet

Defining Variables

In C++, I define a variable by writing its type followed by its name.

```
C++

int age;
double price = 12.5;
bool isReady = true;
char grade = 'A';
string name = "Alex";
```

Common Variable Type

Type	Purpose	Example
int	Whole numbers	int age = 20;
double	Decimal numbers	double price = 3.14;
float	Decimal numbers with less precision	float value = 2.5f;
bool	True or false	bool passes = true;
char	One character	char grade = ‘A’;
string	Text	string name = “Sam”;

To use string:

```
C++

#include <string>
```

Arrays

An array stores multiple values of the same type.

```
C++

int numbers[5];
```

Assigning values:

```
C++

numbers[0] = 10;
numbers[1] = 20;
```

Reading a value:

```
C++

cout << numbers[0];
```

Array indexes begin at 0, so an array of size 5 has indexes 0 to 4.

An array can also be initialized directly:

```
C++

int numbers[5] = {10, 20, 30, 40, 50};
```

2D Arrays

A two-dimensional array stores values in rows and columns.

```
C++

int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Accessing a value:

```
C++

cout << matrix[1][2];
```

This prints 6.

Input and Output

Use cout to print output:

```
C++

cout << "Hello" << endl;
```

Use cin to read input:

```
C++

int age;
cin >> age;
```

Multiple values could be read together:

```cpp
int a;
int b;

cin >> a >> b;
```

Conditions and If statements

An if statement runs code when a condition is true.

```cpp
if (age >= 18) {
    cout << "Adult" << endl;
}
```

Using else:

```cpp
if (score >= 50) {
    cout << "Pass" << endl;
} else {
    cout << "Fail" << endl;
}
```

Using else if:

```cpp
if (score >= 80) {
    cout << "High Distinction" << endl;
} else if (score >= 70) {
    cout << "Distinction" << endl;
} else {
    cout << "Other grade" << endl;
}
```

Common comparison operations:

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical operators:

Operator	Meaning
&&	AND
||	OR
!	NOT

For loops:

A for loop is useful when the number of repetitions is known.

```
C++
for (int i = 0; i < 5; i++) {
    cout << i << endl;
}
```

Reading values into an array:

```C++
for (int i = 0; i < n; i++) {
    cin >> numbers[i];
}
```

While Loops

A while loop continues while its condition is true.

```C++
int i = 0;

while (i < 5) {
    cout << i << endl;
    i++;
}
```

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <string>
using namespace std;

double read_input(string prompt)
{
    int n;
    cout<< prompt << endl;
    cin >> n;
    return n;
}

void fill_array(double arr[], int n)
{
    cout<< "Please enter the elements of the array:" << endl;
    for (int i = 0; i < n; i++)
    {

        cin>>arr[i];
    }
}

void is_sorted(double arr[], int n)
{
    bool is_sorted = true;
    for (int i = 1; i < n; i++)
    {
        if (arr[i] < arr[i - 1])
        {
            is_sorted = false;
        }
    }
    if (is_sorted)
    {
        cout<<"Yes"<<endl;
    }else
    {
        cout<<"No"<<endl;
    }
}

int main()
{
    int n = read_input("Please enter the number of elements you want to sort");
    double arr[n];
    fill_array(arr, n);
    is_sorted(arr,n);
}
```

P1 Report

The command you used to compile the program:
clang++ Subtask2.cpp -o Subtask2

```
yihaoyu@dyn-118-138-83-168 P1 % ls -1
C++_Cheatsheet.docx
C++_Cheatsheet.pdf
Subtask2
Subtask2.cpp

yihaoyu@dyn-118-138-83-168 P1 % clang++ Subtask2.cpp -o Subtask2
```

The command you used to run the program:
./Subtask2

```
yihaoyu@dyn-118-138-83-168 P1 % ./Subtask2
Please enter the number of elements you want to sort
```

Entering the input and the program printing the output:

```
yihaoyu@dyn-118-138-83-168 P1 % ./Subtask2
Please enter the number of elements you want to sort
4
Please enter the elements of the array:
2.1
1.2
2.3
4.3
No
```

```
yihaoyu@dyn-118-138-83-168 P1 % ./Subtask2
Please enter the number of elements you want to sort
4
Please enter the elements of the array:
1
1.2
2.3
4.5
Yes
```

6 Sorting on Hardware

Compare different sorting algorithms and pick the appropriate one(s).

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/08/03 14:29	Yihao Yu	Working On It
2026/08/05 11:07	Yihao Yu	Ready for Feedback
2026/08/05 11:07	Gary Zheng	Tests complete: 1 / 1
2026/08/05 16:39	Gary Zheng	Hi Yihao, thank you for your P2. There is one issue I can notice:It seems that you will never choose anything else other than selection sort based on your existing logic.The reason here is in your insertion sort implementation as you constantly swap elements, while for standard insertion sort it holds the element for sorting in a temp variable and shifts neighbour over. Right now your insertion sort is using a similar logic to bubble sort, which is what makes it inefficient.Finally, for your evidence of testing, notice for both cases your algorithm return the same cost, which is quite redundant, so please test different cases.Let me know if you have any questions.
2026/08/05 16:39	Gary Zheng	Fix and Resubmit
2026/08/11 10:33	Yihao Yu	Ready for Feedback
2026/08/11 10:34	Gary Zheng	Tests complete: 1 / 1
2026/08/11 14:11	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Sorting on Hardware

Submitted By:
Yihao Yu
yyuu0140
2026/08/11 10:33

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

August 11, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <string>

using namespace std;

struct Post {
    string username;
    string type; // will be either "photo" or "video"
    int likes;
    int comments;
    int length_in_seconds; // only applies if it's a video
    int posted_timestamp; // the timestamp of when it was posted
};

int calculate_cost(int mem_write_cost, int mem_read_cost, int mem_store_cost, int write_count, int read_count, int store_count)
{
    return mem_write_cost*write_count + mem_read_cost*read_count + mem_store_cost*store_count;
}

// Selection Sort
int biggest_number_index(Post posts[], int n, int from_index)
{
    int current_biggest_index = from_index;
    for (int i = from_index+1; i<n; i++)
    {
        if (posts[i].likes > posts[current_biggest_index].likes)
        {
            current_biggest_index = i;
        }
        else if (posts[i].likes == posts[current_biggest_index].likes and posts[i].posted_timestamp < posts[current_biggest_index].posted_timestamp)
        {
            current_biggest_index = i;
        }
    }
    return current_biggest_index;
}

void selection_sort(Post posts[], int n)
{
    for (int i = 0; i < n; i++)
    {
        int next_biggest_index = biggest_number_index(posts, n, i);
        swap(posts[i],posts[next_biggest_index]);
    }
}

```

```cpp
int estimate_selection_cost(int n, int mem_write_cost, int mem_read_cost, int mem_store_cost)
{
    int read_count = 2*(n-1) + n * (n - 1);
    int write_count = 2*(n-1);
    int store_count = 0;

    return calculate_cost(mem_write_cost, mem_read_cost, mem_store_cost, write_count, read_count, store_count);
}

// Insertion Sort
/**
void insertion_sort(Post posts[], int n)
{
    for (int first_unsorted_index = 0; first_unsorted_index < n; first_unsorted_index++)
    {
        int current_index = first_unsorted_index;
        while (current_index - 1 >=0 &&
            (posts[current_index].likes > posts[current_index - 1].likes
             || (posts[current_index - 1].likes == posts[current_index].likes &&
              posts[current_index - 1].posted_timestamp >
              posts[current_index].posted_timesta
mp)   ))
        {
            swap(posts[current_index],posts[current_index - 1]);
            current_index --;
        }
    }
}

int estimate_insertion_cost(int n, int mem_write_cost, int mem_read_cost, int mem_store_cost)
{
    int read_count = 2* n * (n-1);
    int write_count = n * (n-1);
    int store_count = 0;

    return calculate_cost(mem_write_cost, mem_read_cost, mem_store_cost, write_count, read_count, store_count);
}
**/

void insertion_sort(Post posts[], int n)
{
    for (int first_unsorted_index = 0; first_unsorted_index < n; first_unsorted_index++)
    {
        int current_index = first_unsorted_index;
        Post temp_post = posts[current_index];
        while (current_index - 1 >=0 &&
                    (temp_post.likes > posts[current_index - 1].likes
```

```
                    || (posts[current_index - 1].likes == temp_post.likes &&
                    ↪  posts[current_index - 1].posted_timestamp >
                    ↪  temp_post.posted_timestamp)   ))
        {
            posts[current_index] = posts[current_index - 1];
            current_index --;
        }

        posts[current_index] = temp_post;

    }
}

int estimate_insertion_cost(int n, int mem_write_cost, int mem_read_cost, int
↪  mem_store_cost)
{
    int read_count = (n-1) + n*(n-1);
    int write_count = (n-1) + n * (n-1)/2;
    int store_count = 0;

    return calculate_cost(mem_write_cost, mem_read_cost, mem_store_cost,
    ↪  write_count, read_count, store_count);
}
// Bubble Sort
void bubble_sort(Post posts[], int n)
{
    for (int pass_counter = 0; pass_counter < n; pass_counter++)
    {
        bool pass_had_swaped = false;

        for (int left_item=0; left_item < n-1; left_item++)
        {
            if ((posts[left_item].likes < posts[left_item+1].likes)
                || (posts[left_item].likes == posts[left_item+1].likes &&
                ↪  posts[left_item].posted_timestamp >
                ↪  posts[left_item+1].posted_timestamp))
            {
                swap(posts[left_item],posts[left_item+1]);
                pass_had_swaped = true;
            }
        }

        if (!pass_had_swaped) break;
    }
}

int estimate_bubble_cost(int n, int mem_write_cost, int mem_read_cost, int
↪  mem_store_cost)
{
    int read_count = 2*n*(n-1) + n * (n - 1);
    int write_count = n*(n-1);
    int store_count = 0;

```

```cpp
    return calculate_cost(mem_write_cost, mem_read_cost, mem_store_cost,
    ↪  write_count, read_count, store_count);
}

// Cost comparison
int minimize_cost(Post posts[], int n, int mem_write_cost, int mem_read_cost, int
↪  mem_store_cost)
{
    int selection_cost = estimate_selection_cost(n, mem_write_cost, mem_read_cost,
    ↪  mem_store_cost);
    int insertion_cost = estimate_insertion_cost(n, mem_write_cost, mem_read_cost,
    ↪  mem_store_cost);
    int bubble_cost = estimate_bubble_cost(n, mem_write_cost, mem_read_cost,
    ↪  mem_store_cost);
    cout << "The cost for Selection sort is: " << selection_cost << endl;
    cout << "The cost for Insertion sort is: " << insertion_cost << endl;
    cout << "The cost for Bubble sort is: " << bubble_cost << endl;
    cout << "So I would choose: ";

    if (selection_cost <= insertion_cost && selection_cost <= bubble_cost)
    {
        selection_sort(posts,n);
        cout << "Selection sort"<<endl;
        return selection_cost;
    }
    else if (insertion_cost <= selection_cost && insertion_cost <= bubble_cost)
    {
        insertion_sort(posts,n);
        cout << "Insertion sort"<<endl;
        return insertion_cost;
    }
    else if (bubble_cost <= selection_cost && bubble_cost <= insertion_cost)
    {
        bubble_sort(posts,n);
        cout << "Bubble sort"<<endl;
        return bubble_cost;
    }
}


int sort_posts(Post posts[], int n, int mem_write_cost, int mem_read_cost, int
↪  mem_store_cost) {
    // ... implement your logic here ...
    // Return the estimated cost
    return minimize_cost(posts, n, mem_write_cost, mem_read_cost, mem_store_cost);
}

int main() {
    Post posts[10];
    for (int i = 0; i < 10; i++) {
        posts[i].username = "Cool user";
        posts[i].type = "photo";
        posts[i].likes = i;
        posts[i].comments = 3 * i;
```

```
186         posts[i].length_in_seconds = 0;
187         posts[i].posted_timestamp = i;
188     }
189 
190     int cost = sort_posts(posts, 10, 1, 20, 1);
191 
192     cout << "Cost of sorting was " << cost << endl;
193     cout << "# likes are:" << endl;
194     for (int i = 0; i < 10; i++) {
195         cout << posts[i].likes << " ";
196     }
197     cout << endl;
198 }
```

P2 Report

Reasoning:

The task requires choosing the most cost-efficient sorting algorithm based on different costs. Selection sort, Insertion sort and Bubble sort have different characteristics in terms of comparisons and swaps, so we need to estimate the cost of them before actual sorting. Accord to the material, we need only focus on the reads/writes performed on the input array.

The estimation uses a simplified cost model. It assumes worst-case behaviour.

For Selection sort,

We need to compare [(n-1) + (n-2) + (n-3) + ... + 1] = (n-1) *n/2 times and we need to swap at most (n-1) times.

Therefore,

```
int read_count = 2*(n-1) + n * (n - 1);
int write_count = 2*(n-1);
```

For Insertion sort,

We use a temporary variable called temp_post to hold the first unsorted elements in the array. Therefore, we need to read (n-1) times. Then, we need to compare and shift [(n-1) + (n-2) + ... + 1] = (n-1) *n/2 times.

Therefore,

```
int read_count = (n-1) + n*(n-1);
int write_count = (n-1) + n * (n-1)/2;
```

For Bubble sort,

We need to compare (n-1)*n times and We need to swap [(n-1) + (n-2)+ ... +1] = (n-1)*n/2 times.

Therefore,

```
int read_count = 2*n*(n-1) + n * (n - 1);
int write_count = n*(n-1);
```

After estimation of each sorting method, we could know which one is the most efficient way. And then choose the most efficient way to deal with this task.

Evidence of Testing:

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P2/P2task
The cost for Selection sort is: 126
The cost for Insertion sort is: 270
The cost for Bubble sort is: 360
So I would choose: Selection sort
Cost of sorting was 126
# likes are:
100 96 92 88 84 80 76 72 68 64
```

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P2/P2task
The cost for Selection sort is: 396
The cost for Insertion sort is: 738
The cost for Bubble sort is: 1440
So I would choose: Selection sort
Cost of sorting was 396
# likes are:
9 8 7 6 5 4 3 2 1 0
```

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P2/P2task
The cost for Selection sort is: 2178
The cost for Insertion sort is: 2034
The cost for Bubble sort is: 5490
So I would choose: Insertion sort
Cost of sorting was 2034
# likes are:
9 8 7 6 5 4 3 2 1 0
```

Word Count: 203.

7 Pointing Linked Arrays

Implement a variant of linked lists, where each node contains more than 1 item.

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/08/05 11:07	Yihao Yu	Working On It
2026/08/11 11:45	Yihao Yu	Ready for Feedback
2026/08/11 11:46	Gary Zheng	Tests complete: 2 / 2
2026/08/11 13:17	Yihao Yu	Ready for Feedback
2026/08/11 13:18	Gary Zheng	Tests complete: 2 / 2
2026/08/11 14:34	Gary Zheng	Discuss
2026/08/11 14:34	Gary Zheng	discussed in class, add test cases then pass.
2026/08/11 14:34	Gary Zheng	Fix and Resubmit
2026/08/11 14:41	Yihao Yu	Ready for Feedback
2026/08/11 14:41	Gary Zheng	Tests complete: 1 / 2
2026/08/11 14:41	Gary Zheng	**Automated comment**: Some tests did not pass for this submission. Please review the Overseer report, verify your output, and resubmit.
2026/08/11 15:02	Yihao Yu	Ready for Feedback
2026/08/11 15:08	Gary Zheng	Tests complete: 1 / 2
2026/08/11 15:08	Gary Zheng	**Automated comment**: Some tests did not pass for this submission. Please review the Overseer report, verify your output, and resubmit.
2026/08/11 15:09	Yihao Yu	Ready for Feedback
2026/08/11 15:10	Gary Zheng	Tests complete: 2 / 2
2026/08/11 18:15	Yihao Yu	I have already add my test cases. Thanks for your patience and replay :)
2026/08/11 18:22	Gary Zheng	Discuss
2026/08/11 18:23	Gary Zheng	Perfect, I can tick you off now. Come see me next week with your QR code.
2026/08/18 14:06	Gary Zheng	Complete
2026/08/18 14:06	Gary Zheng	Discussed in class
2026/08/18 14:06	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Pointing Linked Arrays

Submitted By:
Yihao Yu
yyuu0140
2026/08/11 14:41

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

August 11, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <string>

using namespace std;

const int ARRAY_SIZE = 100;

struct Node {
    // Implement as you need
    int arr[ARRAY_SIZE];
    Node* next;
};

struct PointingLinkedArrays {
    Node* first_node = nullptr;
    int size = 0;
    int node_size = 0;
    int get_size() {
        // ...
        return size;
    }

    // O(n): Each nodes stores 100 integers, so there are (n/100 + 1) nodes.
    // If we want to append the array, we need to travel from the first node to the
    // last node. Therefore, the time complexity is Linear.
    void append(int value) {
        // ...
        if (size%ARRAY_SIZE == 0)
        {
            Node* new_node = new Node();
            new_node->arr[0] = value;
            new_node->next = nullptr;
            if (first_node == nullptr)
            {
                first_node = new_node;
            }else
            {
                Node* current_node = first_node;
                for (int i = node_size-1; i>0; i--)
                {
                    current_node = current_node->next;
                }
                current_node->next = new_node;
            }
            size++;
            node_size++;
            return;
        }
        Node* current_node = first_node;
        for (int i = node_size-1; i > 0; i--)
```

```
        {
            current_node = current_node->next;
        }
        current_node->arr[size%ARRAY_SIZE] = value;
        size++;

    }

    // O(n): If we want to get to the index we want, we need to travel from the first
    // node to the (index/100 + 1)th node. Therefore, the time is directly proportional to
    // the index.
    int get_index(int index) {
        // ...
        int node_number = index/ARRAY_SIZE;
        int index_node = index%ARRAY_SIZE;
        Node* current_node = first_node;
        for (int i = node_number; i > 0; i--)
        {
            current_node = current_node->next;
        }
        return current_node->arr[index_node];
    }
    void set_index(int index, int new_value) {
        // ...
        int node_number = index/ARRAY_SIZE;
        int index_node = index%ARRAY_SIZE;
        Node* current_node = first_node;
        for (int i = node_number; i > 0; i--)
        {
            current_node = current_node->next;
        }
        current_node->arr[index_node] = new_value;
    }
};

int main() {
    PointingLinkedArrays list;

    string menu = "1. Append a new item\n2. Get value of an index\n3. Set value of an index\n4. See the current size\n5. See the menu again\n6. Quit";
    int command = 5;

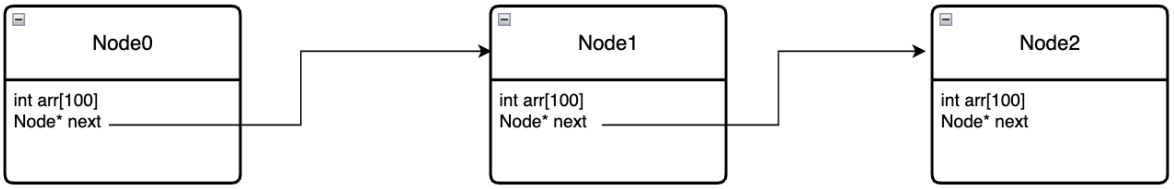
    while (command != 6) {
        if (command == 1) {
            cout << "Enter the value to append:" << endl;
            int new_val;
            cin >> new_val;
            list.append(new_val);
        } else if (command == 2) {
            cout << "Enter the index:" << endl;
            int index;
            cin >> index;
```

```
102             cout << "Value at index " << index << ": " << list.get_index(index) <<
                ↪  endl;
103         } else if (command == 3) {
104             cout << "Enter the index:" << endl;
105             int index;
106             cin >> index;
107             cout << "Enter the new value:" << endl;
108             int new_value;
109             cin >> new_value;
110             list.set_index(index, new_value);
111         } else if (command == 4) {
112             cout << "Current size: " << list.get_size() << endl;
113         } else if (command == 5) {
114             cout << menu << endl;
115         }
116 
117         cout << "Enter command (1-6):" << endl;
118         cin >> command;
119     }
120 
121     return 0;
122 }
```

P3 Report

Visualisation:

SID:082



Evidence of Testing

```
1. Append a new item
2. Get value of an index
3. Set value of an index
4. See the current size
5. See the menu again
6. Quit
Enter command (1-6):
1
Enter the value to append:
1
Enter command (1-6):
1
Enter the value to append:
2
Enter command (1-6):
1
Enter the value to append:
3
Enter command (1-6):
1
Enter the value to append:
4
Enter command (1-6):
1
Enter the value to append:
5
Enter command (1-6):
1
Enter the value to append:
6
```

```
1. Append a new item
2. Get value of an index
3. Set value of an index
4. See the current size
5. See the menu again
6. Quit
Enter command (1-6):
4
Current size: 21
Enter command (1-6):
2
Enter the index:
1
Value at index 1: 2
Enter command (1-6):
2
Enter the index:
0
Value at index 0: 1
Enter command (1-6):
2
Enter the index:
3
Value at index 3: 4
```

```
Enter command (1-6):
5
1. Append a new item
2. Get value of an index
3. Set value of an index
4. See the current size
5. See the menu again
6. Quit
Enter command (1-6):
4
Current size: 21
Enter command (1-6):
3
Enter the index:
0
Enter the new value:
1
Enter command (1-6):
5
1. Append a new item
2. Get value of an index
3. Set value of an index
4. See the current size
5. See the menu again
6. Quit
Enter command (1-6):
2
Enter the index:
0
Value at index 0: 1
```

```
1. Append a new item
2. Get value of an index
3. Set value of an index
4. See the current size
5. See the menu again
6. Quit
Enter command (1-6):
2
Enter the index:
0
Value at index 0: 1
Enter command (1-6):
2
Enter the index:
1
Value at index 1: 2
Enter command (1-6):
3
Enter the index:
0
Enter the new value:
5
Enter command (1-6):
5
1. Append a new item
2. Get value of an index
3. Set value of an index
4. See the current size
5. See the menu again
6. Quit
Enter command (1-6):
2
Enter the index:
0
Value at index 0: 5
```

```
Enter command (1-6):
4
Current size: 200
Enter command (1-6):
2
Enter the index:
130
Value at index 130: 130
```

8 Complex Complexities

Analyse the time complexity of a few functions, measure their runtime and write a report.

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/08/11 10:36	Yihao Yu	Working On It
2026/08/18 11:44	Yihao Yu	Ready for Feedback
2026/08/18 14:45	Gary Zheng	discussed in class
2026/08/18 14:45	Gary Zheng	Fix and Resubmit
2026/08/18 15:34	Yihao Yu	Ready for Feedback
2026/08/20 21:07	Gary Zheng	Hi Yihao, this looks good now.Happy to tick it off.
2026/08/20 21:07	Gary Zheng	Complete
2026/08/20 22:52	Yihao Yu	Thank you very much! I really appreciate your labour and work!

Monash

Fundamentals of Algorithms

OnTrack Submission

Complex Complexities

Submitted By:
Yihao Yu
yyuu0140
2026/08/18 15:34

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

August 18, 2026



Operations and Complexity

Function bubble_sort

Number of operations is n*(n-1) / 2, because when it comes to comparison in the worst case, the inner loop becomes shorter after every pass. Then we will get:

(n-1) + (n-2) + ... + 1

By using arithmetic-series formula:
(n-1)*n / 2

The Big-O complexity is $O(n^2)$ because the dominant term in (n-1)*n / 2 is n^2 .

Function user_score

Number of operations is constant because the function only contains a fixed number of if statement and arithmetic operations. It doesn't have any loops, and the amount of array do not depend on the size of array.

The Big-O complexity is O(1) because the same fixed number of operations is performed regardless of the input size.

Function linear_search

Number of operations is n in the worst case because the target may either be the last element in the array do not exist in the array.

The Big-O complexity is O(n) because the number of operations grows linearly with the size of array.

Function count_liked

Number of operations is l*(c + s) because the loop runs l times and for each loop it performs two linear searches. The number of operations is approximately l*(c + s).

The Big-O complexity is O (l*c +s*l)because when l, c and s are approaching infinite, then O(l*(c + s)) = O (l*c +s*l)

Function binary_search

Number of operations is $\log_2 n$ because each iteration removes half of the remaining array.

The Big-O complexity is O (log n) because the number of operations is $\log_2 n$. The search space is divided by 2 during each iteration.

Function count_comment_no_like

Number of operations is $c*(\log_2 l + s)$ because the outer loop runs c times, and for every comment would be executed by binary search and if user doesn't like the post, it would be executed by linear search as well.

The Big-O complexity is O (c * log l + c * s)because when l, c and s are approaching infinite, then O ($c*(\log_2 l + s)$) = O (c * log l + c * s)

Function count_only_share

Number of operations is $s*(\log_2 l+c)$ because the outer loop runs s times, and for every share would be executed by binary search and linear search.
The Big-O complexity is O (s * log l + s**2) because when l, c and s are approaching infinite, then O ($s*(\log_2 l + c)$) = O (s * log l + s*c)

Function `engagement_score`

Number of operations is $(l-1) *l / 2 + l*(c + s) + c*(\log_2 l + s) + s*(\log_2 l+c)$ because the function firstly calls bubble_sort. It then calls count_liked, count_comment_no_liked and count_only_share.

The Big-O complexity is:
$O(l^2 / 2 – l / 2 + l*c + l*s + c*\log l + 2*c*s + s*\log l)$
$= O(l^2 / 2 – l /2 + l * (c+s) + (c+s)*\log l + 2*c*s)$
$= O (l^2 /2 + l * (c +s – ½) + (c+s)*\log l +2 *c *s)$
$= O (l^2+ l * (c +s) + (c + s)*\log l + c*s)$
$= O (l^2 + c*l + s*l + c*s)$

Execution time was measured on input sizes 100, 500, 1000, 2000, 4000 and the results were 615 ms, 13556 ms, 45177 ms, 93090 ms, 370084 ms.

Evidence of Measurement

```
void test_execution_time(int n) {
    string* likes = new string[n];
    string* comments = new string[n];
    string* shares = new string[n];

    for (int i = 0; i < n; i++) {
        likes[i] = "user" + to_string(i);
        comments[i] = "user" + to_string(i);
        shares[i] = "user" + to_string(i);
    }

    auto start :time_point<steady_clock, steady_clock::duration> = chrono::high_resolution_clock::now();

    int score = engagement_score(
        likes, l: n,
        comments, c: n,
        shares, s: n
    );

    auto end :time_point<steady_clock, steady_clock::duration> = chrono::high_resolution_clock::now();

    auto duration =
        chrono::duration_cast<chrono::microseconds>( fd: end - start);

    cout << "Input size n = " << n
         << ", Execution time = "
         << duration.count()
         << " microseconds"
         << endl;

    delete[] likes;
    delete[] comments;
    delete[] shares;
}
```

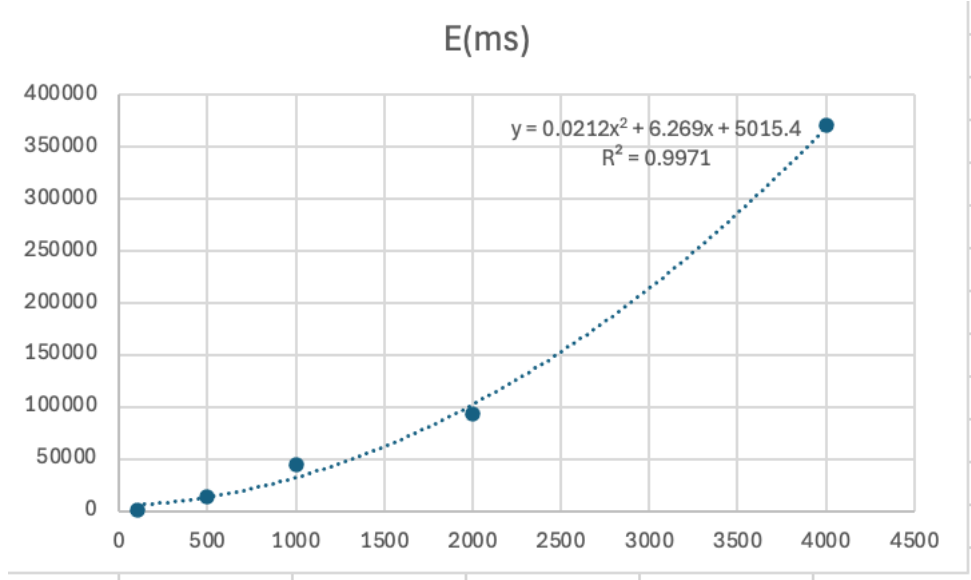
```cpp
int main() {
    /*
    string likes[4] = {"Nora", "Ethan", "Maya", "Liam"};
    string comments[3] = {"Ethan", "Maya", "Zara"};
    string shares[3] = {"Nora", "Maya", "Oscar"};
    int l = 4;
    int c = 3;
    int s = 3;
    int score = engagement_score(likes, l, comments, c, shares, s);
    cout << "Engagement score for this post is " << score << endl;
    return 0;
    */

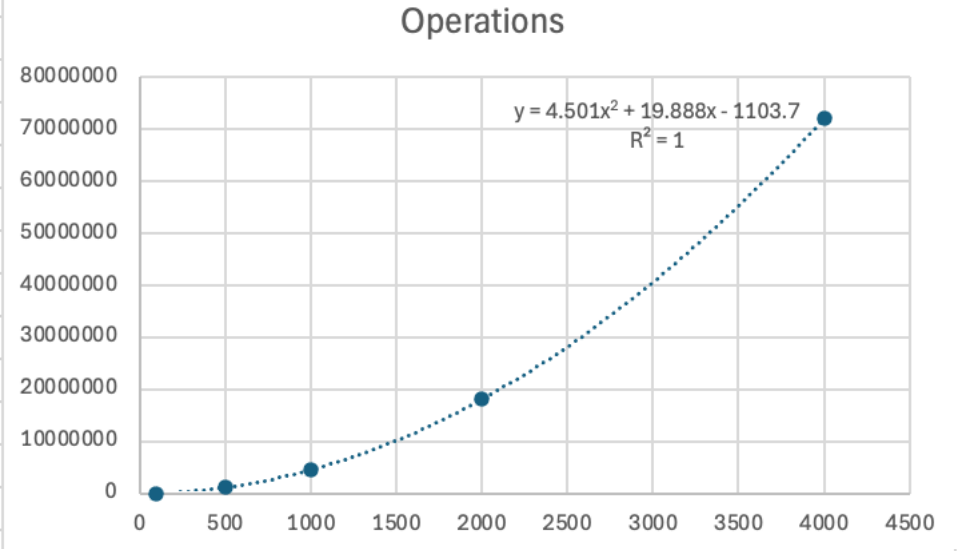
    test_execution_time(n:100);
    test_execution_time(n:500);
    test_execution_time(n:1000);
    test_execution_time(n:2000);
    test_execution_time(n:4000);
}
```

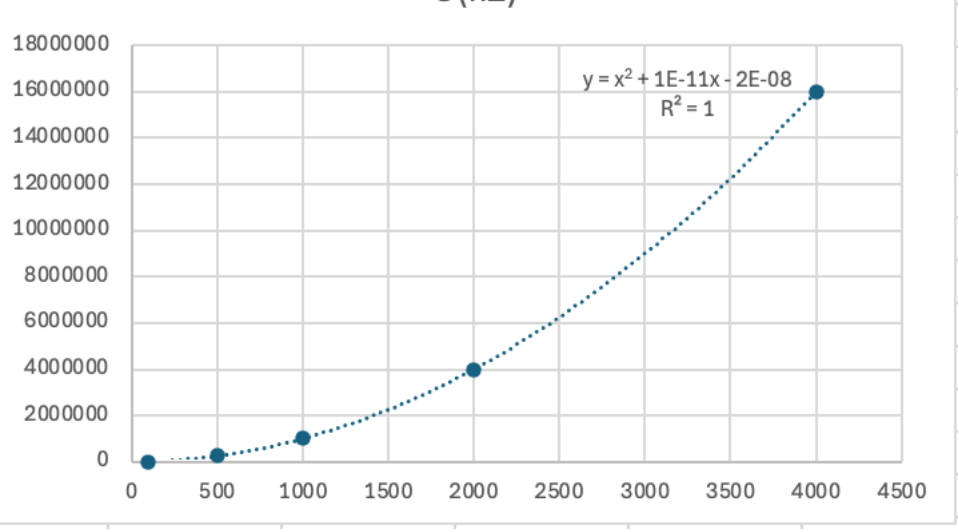
```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P4/P4_task
Input size n = 100, Execution time = 615 microseconds
Input size n = 500, Execution time = 13556 microseconds
Input size n = 1000, Execution time = 45177 microseconds
Input size n = 2000, Execution time = 93090 microseconds
Input size n = 4000, Execution time = 370084 microseconds

Process finished with exit code 0
```

Charts


E(ms)
y = 0.0212x² + 6.269x + 5015.4
R² = 0.9971
400000
350000
300000
250000
200000
150000
100000
50000
0
0
500
1000
1500
2000
2500
3000
3500
4000
4500


Operations
y = 4.501x² + 19.888x - 1103.7
R² = 1
80000000
70000000
60000000
50000000
40000000
30000000
20000000
10000000
0
0
500
1000
1500
2000
2500
3000
3500
4000
4500


O(n2)
y = x² + 1E-11x - 2E-08
R² = 1
18000000
16000000
14000000
12000000
10000000
8000000
6000000
4000000
2000000
0
0
500
1000
1500
2000
2500
3000
3500
4000
4500

Conclusion

If a different input were sorted instead of likes, the final big-O notation would still be O (l^2 + c*l + s*l + c*s). This is mainly because bubble sort itself has a complexity of O (l^2 + c*l + s*l + c*s), and the remaining functions still contain combination of loops and linear search produces quadratic behaviour. The actual execution time depends on the relative size of likes, comments and shares and how often each array is searched.

(l-1) *l / 2 + l*(c + s) + c*($\log_2$l + s) + s*($\log_2$l+c)
= l^2/2 – l/2 + l*c + l*s + 2 * c*s + (c+s)*$\log_2$l
= l^2/2 – l/2 + 2 * c*s + (c+s)*($\log_2$l + l)

The graph shows the measured execution time literally follows the same increasing trend as the estimated number of operations and theoretical complexity. As the input increases, all three lines grow at a similar quadratic rate.

Word Count

This document contains 483 words.

9 Stacks & Queues

Implement Stacks & Queues, and determine when one is appropriate to use

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/08/18 14:08	Yihao Yu	Working On It
2026/08/21 01:05	Yihao Yu	Ready for Feedback
2026/08/21 01:05	Gary Zheng	Tests complete: 3 / 3
2026/08/22 17:21	Gary Zheng	Hi Yihao, P5 looks good, happy to tick it off.Only point of feedback is:Your queue testing drains to empty but never enqueues again afterwards, to test if front and rear is correct, you could try adding a few more items after - of course this is not a big issue.
2026/08/22 17:21	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Stacks & Queues

Submitted By:
Yihao Yu
yyuu0140
2026/08/21 01:05

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

August 21, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <string>

using namespace std;

struct Node {
    Node* next = nullptr;
    int value;
};

struct StrNode {
    StrNode* next = nullptr;
    string value;
};

struct Stack {
    Node* top = nullptr;
    int size = 0;

    void push(int value) {
        Node* new_node = new Node;
        new_node->value = value;
        new_node->next = top;
        top = new_node;
        size++;
    }

    int peek() {
        if (top == nullptr) {
            throw "Stack is empty";
        }
        return top->value;
    }

    int pop() {
        if (top == nullptr) {
            throw "Stack is empty";
        }
        int result = top->value;
        Node* new_top = top->next;
        delete top;
        top = new_top;
        size--;
        return result;
    }

    // TASK 1
    int current_size() {
        // TODO
        return size;
```

```
    }
};

struct LinkedQueue {
    // TASK 2: Implement the linked queue class.
    // You should be able to use one of the Node structs above.
    // Recall that a queue requires two pointers, a `rear` and a `front`.
    StrNode* front = nullptr;
    StrNode* rear = nullptr;
    int size = 0;

    void enqueue(string item) {
        StrNode* new_node = new StrNode;
        new_node->value = item;
        new_node->next = nullptr;

        size++;

        if (is_empty())
        {
            front = new_node;
            rear = new_node;
        } else
        {
            rear->next = new_node;
            rear = new_node;
        }

    }

    string dequeue() {
        if (is_empty())
        {
            throw "Queue is empty";
        }

        StrNode* next_front = front->next;
        string result = front->value;
        delete front;
        front = next_front;
        size--;

        if (is_empty())
        {
            rear = nullptr;
        }
        return result;


    }

    string peek() {
        if (is_empty())
        {
```

```
105             throw "Queue is empty";
106         }
107         string result = front->value;
108         return result;
109     }
110 
111     bool is_empty() {
112         return front == nullptr;
113     }
114 
115 };
116 
117 int main() {
118     Stack stack;
119     LinkedQueue queue;
120 
121     string menu = "1. Push a value onto the stack\n2. Pop the top value off the
    ↪  stack\n3. Peek at the top of the stack\n4. Enqueue a value into the
    ↪  queue\n5. De
122 queue the front value from the queue\n6. Peek at the front of the queue\n7. See the
    ↪  current stack size\n8. See the menu again\n9. Quit";
123 
124     cout << menu << endl;
125     int command = 0;
126     cout << "Enter command (1-9):" << endl;
127     cin >> command;
128 
129     while (command != 9) {
130         // The stack/queue methods throw when they are empty, so we wrap
131         // each action in a try/catch and report the error instead of crashing.
132         try {
133             if (command == 1) {
134                 cout << "Enter the value to push:" << endl;
135                 int value;
136                 cin >> value;
137                 stack.push(value);
138             } else if (command == 2) {
139                 cout << "Popped: " << stack.pop() << endl;
140             } else if (command == 3) {
141                 cout << "Top of stack: " << stack.peek() << endl;
142             } else if (command == 4) {
143                 cout << "Enter the value to enqueue:" << endl;
144                 string value;
145                 cin >> value;
146                 queue.enqueue(value);
147             } else if (command == 5) {
148                 cout << "Dequeued: " << queue.dequeue() << endl;
149             } else if (command == 6) {
150                 cout << "Front of queue: " << queue.peek() << endl;
151             } else if (command == 7) {
152                 cout << "Stack size: " << stack.current_size() << endl;
153             } else if (command == 8) {
154                 cout << menu << endl;
155             } else {
```

```
156                 cout << "Unknown command." << endl;
157             }
158         } catch (const char* error) {
159             cout << "Error: " << error << endl;
160         }
161 
162         cout << "Enter command (1-9):" << endl;
163         cin >> command;
164     }
165 
166     return 0;
167 }
```

P5 Report

Scenario Justification

Scenario 1 - The waiting list for seats to a show

A queue is the most suitable data structure for a waiting list because it follows the FIFO, principle. The person who joins the waiting list first should have the most priority so that everyone would have fair allocation. Stack would be inappropriate because it follows LIFO, meaning most recently added the person would receive the seat first.

Scenario 2 - List of followers of a user on TipTop

Neither a queue nor a stack. Users do not genuinely need to be processed in LIFO or FIFO order. A stack or queue would impose an unnecessary ordering rule and make accessing a user inefficiently. A more suitable structure would be a array, list or another collection.

Scenario 3 - Browser History

A stack is suitable for browser history because the back button should return to the most recently visited page first. This follows the LIFO principle. Each newly visited page can be pushed onto the stack, and pressing back can pop the current page to return to the previous one. Queue would be unsuitable because FIFO would return to the oldest visited page first instead of the most recently visited page.

Evidence of Testing

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P5/P5_task
1. Push a value onto the stack
2. Pop the top value off the stack
3. Peek at the top of the stack
4. Enqueue a value into the queue
5. Dequeue the front value from the queue
6. Peek at the front of the queue
7. See the current stack size
8. See the menu again
9. Quit
Enter command (1-9):
1
Enter the value to push:
1
Enter command (1-9):
1
Enter the value to push:
2
Enter command (1-9):
1
Enter the value to push:
3
```

```
Enter the value to push:
4
Enter command (1-9):
1
Enter the value to push:
5
Enter command (1-9):
1
Enter the value to push:
6
```

```
1. Push a value onto the stack
2. Pop the top value off the stack
3. Peek at the top of the stack
4. Enqueue a value into the queue
5. Dequeue the front value from the queue
6. Peek at the front of the queue
7. See the current stack size
8. See the menu again
9. Quit
Enter command (1-9):
7
Stack size: 6
Enter command (1-9):
3
Top of stack: 6
```

```
Enter command (1-9):
2
Popped: 6
Enter command (1-9):
2
Popped: 5
Enter command (1-9):
2
Popped: 4
Enter command (1-9):
2
Popped: 3
Enter command (1-9):
2
Popped: 2
Enter command (1-9):
2
Popped: 1
Enter command (1-9):
2
Popped: Error: Stack is empty
```

```
Enter command (1-9):
4
Enter the value to enqueue:
1
Enter command (1-9):
4
Enter the value to enqueue:
2
Enter command (1-9):
4
Enter the value to enqueue:
3
Enter command (1-9):
4
Enter the value to enqueue:
4
Enter command (1-9):
4
Enter the value to enqueue:
5
```

```
Enter command (1-9):
4
Enter the value to enqueue:
6
```

```
1. Push a value onto the stack
2. Pop the top value off the stack
3. Peek at the top of the stack
4. Enqueue a value into the queue
5. Dequeue the front value from the queue
6. Peek at the front of the queue
7. See the current stack size
8. See the menu again
9. Quit
Enter command (1-9):
6
Front of queue: 1
```

```
Enter command (1-9):
5
Dequeued: 1
Enter command (1-9):
5
Dequeued: 2
Enter command (1-9):
5
Dequeued: 3
Enter command (1-9):
5
Dequeued: 4
Enter command (1-9):
5
Dequeued: 5
Enter command (1-9):
5
Dequeued: 6
Enter command (1-9):
5
Dequeued: Error: Queue is empty
```

10 Templates

Learn about templates, implement a generic sort, and a generic linked list.

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/08/22 13:18	Yihao Yu	Working On It
2026/08/25 11:22	Yihao Yu	Ready for Feedback
2026/08/25 11:23	Gary Zheng	Tests complete: 2 / 2
2026/08/26 22:46	Gary Zheng	Hi Yihao, your C1 looks great, you are using templates correctly so happy to tick this off.
2026/08/26 22:46	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Templates

Submitted By:
Yihao Yu
yyuu0140
2026/08/25 11:22

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

August 25, 2026



```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

struct Rectangle {
    // ...
    int width;
    int height;

    bool operator<(const Rectangle& right) const
    {
        if (width*height < right.width*right.height)
        {
            return true;
        }else
        {
            return false;
        }
    }

    bool operator<=(const Rectangle& right) const
    {
        if (width*height <= right.width*right.height)
        {
            return true;
        }else
        {
            return false;
        }
    }
};

template <typename T>
void insertion_sort(T arr[], int size ) {
    // ...
    for (int first_unsorted_index = 0; first_unsorted_index < size;
    ↪ first_unsorted_index++)
    {
        int current_index = first_unsorted_index;
        T temp = arr[current_index];
        while (current_index -1 >= 0 && temp <= arr[current_index - 1])
        {
            arr[current_index] = arr[current_index - 1];
            current_index--;
        }

        arr[current_index] = temp;

    }

}

```

```cpp
template <typename T>
void merge_sort(T arr[], int n) {
    // ...
    if (n<=1) return;

    int left_have_size = n / 2;
    int right_have_size = n - left_have_size;

    T* left_half = new T[left_have_size];
    T* right_half = new T[right_have_size];

    for (int i = 0; i < n; i++)
    {
        if (i < left_have_size)
        {
            left_half[i] = arr[i];
        }else
        {
            right_half[i-left_have_size] = arr[i];
        }
    }

    merge_sort(left_half, left_have_size);
    merge_sort(right_half, right_have_size);

    int left_half_index = 0;
    int right_half_index = 0;
    int result_index = 0;

    while (left_half_index < left_have_size && right_half_index < right_have_size)
    {
        if (left_half[left_half_index] < right_half[right_half_index])
        {
            arr[result_index] = left_half[left_half_index];
            left_half_index++;
        }else
        {
            arr[result_index] = right_half[right_half_index];
            right_half_index++;
        }
        result_index++;
    }

    while (left_half_index < left_have_size)
    {
        arr[result_index] = left_half[left_half_index];
        left_half_index++;
        result_index++;
    }

    while (right_half_index < right_have_size)
    {
        arr[result_index] = right_half[right_half_index];
        right_half_index++;
```

```
108         result_index++;
109     }
110 
111     delete[] left_half;
112     delete[] right_half;
113 }
114 
115 int main() {
116     srand(time(nullptr));
117     Rectangle rects_1[10], rects_2[10];
118 
119     for (int i = 0; i < 10; i++) {
120         rects_1[i].width = rand() % 20;
121         rects_1[i].height = rand() % 20;
122         rects_2[i].width = rects_1[i].width;
123         rects_2[i].height = rects_1[i].height;
124     }
125 
126     insertion_sort(rects_1, 10);
127     merge_sort(rects_2, 10);
128     int last_area = -1;
129 
130     for (int i = 0; i < 10; i++) {
131         int cur_area = rects_1[i].width * rects_1[i].height;
132         if (cur_area < last_area) {
133             cout << "The sorted arrays don't match!" << endl;
134             return 1;
135         }
136         last_area = cur_area;
137     }
138 
139     for (int i = 0; i < 10; i++) {
140         if (rects_1[i].width != rects_2[i].width || rects_1[i].height !=
        ↪  rects_2[i].height) {
141             cout << "The sorted arrays don't match!" << endl;
142             return 1;
143         }
144     }
145 
146     cout << "Everything seems fine : )" << endl;
147     return 0;
148 }
149 
150 
```

```cpp
#include <iostream>
#include <string>
using namespace std;

template<typename T>
struct Node {
    // ...
    T item;
    Node<T>* next;
};

template<typename T>
struct LinkedList {
    Node<T>* first_node = nullptr;

    int size = 0;

    // TODO: determine what should the input of the method be and implement it
    void append(T item) {
        // ...
        Node<T>* new_node = new Node<T>;
        new_node->item = item;
        new_node->next = nullptr;

        if (first_node == nullptr)
        {
            first_node = new_node;
            size++;
            return;
        }

        Node<T>* current_node = first_node;
        while (current_node->next != nullptr)
        {
            current_node = current_node->next;
        }

        current_node->next = new_node;
        size++;
        return;
    }

    Node<T>* get_node_at_index(int index) {
        // ...
        Node<T>* current_node = first_node;
        for (int i = 0; i < index; i++)
        {
            current_node = current_node->next;
        }
        return current_node;
    }

    // TODO: determine what should the input of the method be and implement it
    void set_index(int index, T item) {
```

```
        // ...
        Node<T>* current_node = get_node_at_index(index);
        current_node->item = item;
        return;
    }

    void display() {
        // This method is not marked (it's optional)
        // But it will be very helpful for your debugging to have a method that
        // prints what's in the linked list currently.
        Node<T>* current_node = first_node;
        while (current_node != nullptr)
        {
            cout << current_node->item << " ";
            current_node = current_node->next;
        }
        cout << endl;
        return;
    }
};


/// Test the linked list implementation
int main() {
    LinkedList<int> list;
    list.append(10);
    list.append(20);
    list.append(30);

    cout << "Linked List: ";
    list.display();

    list.set_index(1, 25);
    cout << "Linked List after setting index 1 to 25: ";
    list.display();

    cout << "Item at index 2: " << list.get_node_at_index(2)->item << endl;

    LinkedList<string> list_string;
    list_string.append("Apple");
    list_string.append("Banana");
    list_string.append("Cat");

    cout << "Linked List: ";
    list_string.display();

    list_string.set_index(1, "Peach");
    cout << "Linked List after setting index 1 to Peach: ";
    list_string.display();

    cout << "Item at index 2: " << list_string .get_node_at_index(2)->item << endl;

    LinkedList<double> list_double;
    list_double.append(10.0);
```

```
109     list_double.append(11.0);
110     list_double.append(12.0);
111 
112     cout << "Linked List: ";
113     list_double.display();
114 
115     list_double.set_index(1, 11.5);
116     cout << "Linked List after setting index 1 to 11.5: ";
117     list_double.display();
118 
119     cout << "Item at index 2: " << list_double .get_node_at_index(2)->item << endl;
120 
121 
122 
123 
124 
125     return 0;
126 }
127 
```

Report C1

Reasoning

I defined < and <= in for the Rectangle. Firstly, we need to compare the area of the Rectangle, but the compiler could not know its semantic meaning. Therefore, I defined < as an approach to let the compiler know what the actual comparison is. Meanwhile, I defined <= as well, since in the merge_sort, the circumstance when the left_half[left_half_index] == right_half[right_half_index] is included in the else.

```
while (left_half_index < left_have_size && right_half_index < right_have_size)
{
    if (left_half[left_half_index] < right_half[right_half_index])
    {
        arr[result_index] = left_half[left_half_index];
        left_half_index++;
    }else
    {
        arr[result_index] = right_half[right_half_index];
        right_half_index++;
    }
    result_index++;
}
```

That means the when the left one equals to the right one, we would choose the right one priorly. In this case, <= is used to ensure the right one is always switched into left.

Evidence of Testing

Test	Screen Shoot
generic_sort.cpp	/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/C1/generic_sort Everything seems fine :) Process finished with exit code 0
generic_linked_list.cpp int test	/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/C1/generic_linked_list Linked List: 10 20 30 Linked List after setting index 1 to 25: 10 25 30 Item at index 2: 30
generic_linked_list.cpp string test	Linked List: Apple Banana Cat Linked List after setting index 1 to Peach: Apple Peach Cat Item at index 2: Cat
generic_linked_list.cpp double test	Linked List: 10 11 12 Linked List after setting index 1 to 11.5: 10 11.5 1 Item at index 2: 12

11 Custom Merge Sort

Implement a variant of merge sort, where we don't know the number of sublists in advance.

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/08/21 01:10	Yihao Yu	Working On It
2026/08/22 13:15	Yihao Yu	Ready for Feedback
2026/08/22 13:16	Gary Zheng	Tests complete: 2 / 2
2026/08/25 14:36	Gary Zheng	discussed in class
2026/08/25 14:36	Gary Zheng	Discuss
2026/08/25 14:37	Gary Zheng	Complete
2026/08/25 14:37	Gary Zheng	Discussed in class

Monash

Fundamentals of Algorithms

OnTrack Submission

Custom Merge Sort

Submitted By:
Yihao Yu
yyuu0140
2026/08/22 13:15

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

August 22, 2026



```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

/// This function simulates giving us the number of available servers.
/// Ignore its implementation, all you need to know is that when you call
/// this function it returns an integer greater than 1. And you may assume it
/// does so in O(1) time.
/// No other assumptions can be made about this function.
int available_servers() {
    return 2 + rand() % 100;
}

/// `sublists` is the arrays that need to be merged
/// `sublist_sizes` is the size of each of those arrays (i.e. sublists[i] will have sublist_sizes[i] items)
/// `number_of_sublists` is the number of sublists passed to the function in the previous 2 parameters
int* merge_sorted_lists(int** sublists, int* sublist_sizes, int number_of_sublists) {
    // Implement the merging logic, return a pointer to the merged array on the heap

    // Calculate the total size of the merged list
    int total_size = 0;

    for (int i = 0; i < number_of_sublists; i++)
    {
        total_size += sublist_sizes[i];
    }

    // Allocate the merged list on the heap
    int* merged_list = new int[total_size];

    // Every sublists would have a index tracker
    int* indexes = new int[number_of_sublists];

    for (int i = 0; i < number_of_sublists; i++)
    {
        indexes[i] = 0;
    }

    // sorted_index is used to track the index =
    int sorted_index = 0;

    for (int i = 0; i < number_of_sublists; i++)
    {
        // First directly copy it
        if (sorted_index == 0)
        {

            for (int j = 0; j < sublist_sizes[i]; j++)
            {
```

```
                merged_list[sorted_index] = sublists[i][j];
                sorted_index++;
            }
            continue;

        }

        int temp_index = 0;
        int temp_size = sorted_index;
        int* temp_sorted_list = new int[sorted_index];

        // Copy the current merged list
        for (int j = 0; j < temp_size; j++)
        {
            temp_sorted_list[j] = merged_list[j];
        }

        // Rewrite the current merged list from index 0
        sorted_index = 0;

        while (temp_index < temp_size && indexes[i] < sublist_sizes[i])
        {
            if (temp_sorted_list[temp_index] < sublists[i][indexes[i]])
            {
                merged_list[sorted_index] = temp_sorted_list[temp_index];
                temp_index++;
            }else
            {
                merged_list[sorted_index] = sublists[i][indexes[i]];
                indexes[i]++;
            }
            sorted_index++;
        }

        while (temp_index < temp_size)
        {
            merged_list[sorted_index] = temp_sorted_list[temp_index];
            sorted_index++;
            temp_index++;
        }

        while (indexes[i] < sublist_sizes[i])
        {
            merged_list[sorted_index] = sublists[i][indexes[i]];
            sorted_index++;
            indexes[i]++;
        }
        delete[] temp_sorted_list;
    }

    delete[] indexes;

    return merged_list;
}
```

```cpp

int* merge_sort(int* items, int n) {
    // Ask for the next k (the number of servers currently available)
    int k = available_servers();
    if (k > n) k = n;

    // Implement the sorting logic, return a pointer to the sorted array on the heap

    //base case
    if (n<=1)
    {
        int* result= new int[n];
        if (n==1)
        {
            result[0] = items[0];
        }
        return result;
    }

    //Split and copy the sublist
    int** sublists = new int*[k];
    int number_of_sublists = k;
    int* sublists_sizes = new int[number_of_sublists];

    int base_size = n/k;
    int reminder = n%k;

    for (int i = 0; i < number_of_sublists; i++)
    {
        sublists_sizes[i] = base_size;

        if (i <reminder)
        {
            sublists_sizes[i]++;
        }

        sublists[i] = new int[sublists_sizes[i]];
    }

    int temp_index = 0;

    for (int i = 0; i < k; i++)
    {
        for (int j = 0; j < sublists_sizes[i]; j++)
        {
            sublists[i][j] = items[temp_index];
            temp_index++;
        }
    }

    //
    for (int i = 0; i < k; i++)
    {
        int* sorted_sublist = merge_sort(sublists[i],sublists_sizes[i]);
```

```
160
161         delete[] sublists[i];
162
163         sublists[i] = sorted_sublist;
164
165     }
166
167     int* result = merge_sorted_lists(sublists, sublists_sizes, k);
168
169     for (int i = 0; i < k; i++)
170     {
171         delete[] sublists[i];
172     }
173
174     delete[] sublists;
175     delete[] sublists_sizes;
176
177     return result;
178 }
179
180 int main(int argc, char** argv) {
181
182     bool data_from_console = argc == 1; // run './a.out' and './a.out bottom text'
        ↪  to see the difference
183
184     int* sorted;
185     int n;
186
187     if (data_from_console) {
188
189         srand(10001);
190
191         cout << "Enter number of data values:" << endl;
192         cin >> n;
193         int* data = new int[n];
194         cout << "Enter data values:" << endl;
195         for (int i = 0; i < n; i++) {
196             cin >> data[i];
197         }
198         sorted = merge_sort(data, n);
199         delete[] data;
200     } else {
201         srand(time(nullptr));
202         n = 10;
203         int data[] = {9, 4, 7, 2, 1, 6, 8, 3, 5, 0};
204
205         sorted = merge_sort(data, n);
206     }
207
208     cout << "Sorted values:" << endl;
209     for (int i = 0; i < n; i++) {
210         cout << sorted[i] << " ";
211         if ((i & 31) == 31) cout << '\n'; // Break up lines so that line by line
            ↪  diff is useful
```

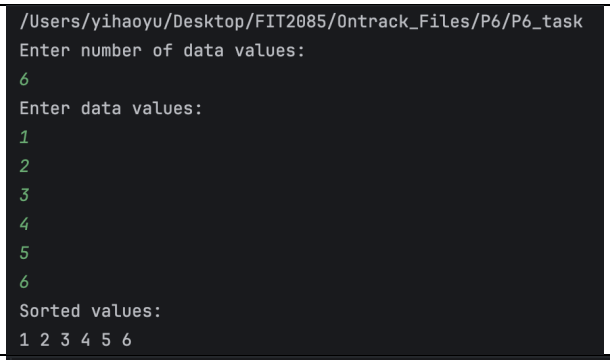
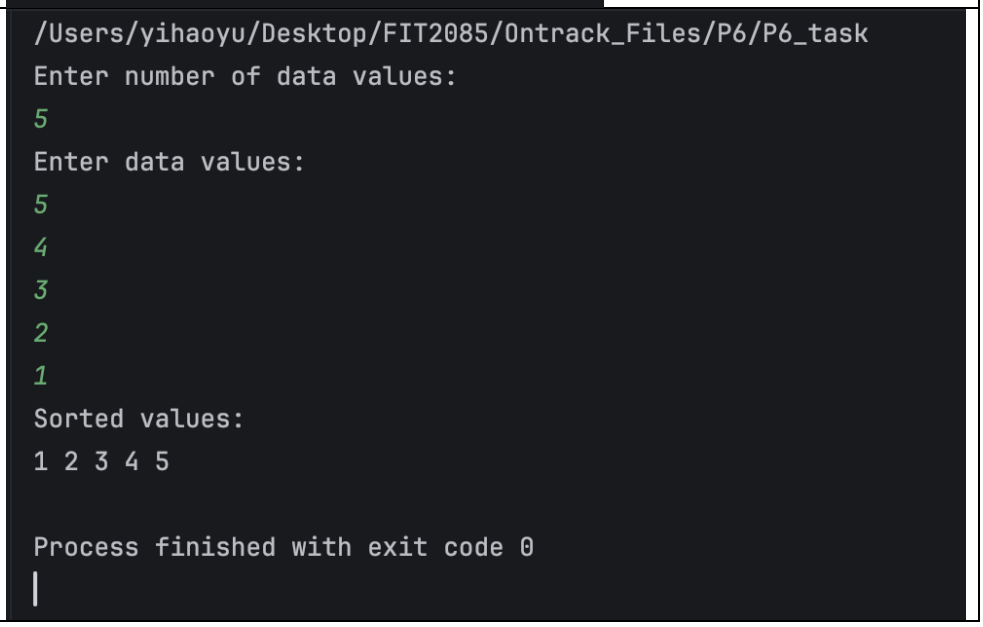
```
212     }
213     cout << endl;
214 
215     return 0;
216 }
```

Report Scaffold

Reasoning

Firstly, my algorithm gets the number of available servers, and limits it to n if k>n. The input array is then divided into k sublists as evenly as possible, so their sizes differ by at most 1. Each sublist is recursively sorted by using the same merge_sort function. The base case occurs when the sublist contains only one element. Subsequently, merge_sorted_lists is used to combine the sorted sublists. In the merge_sorted_lists, I merge sort the sublists sequentially: the first list becomes the initial merged result, then each sublists remained is merged with current result using standard two pointer merge technique.

Evidence of Testing

Test Case	Proof
Already sorted	
Reserve order	

Duplication	/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P6/P6_task Enter number of data values: 5 Enter data values: 1 2 3 3 0 Sorted values: 0 1 2 3 3
Negative numbers	/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P6/P6_task Enter number of data values: 5 Enter data values: -1 -2 -3 -4 -5 Sorted values: -5 -4 -3 -2 -1

Time Complexity Analysis

Let n be the number of elements and k be the number of available servers at a recursive call. The input is divided into approximately equal sublists of size n/k.
My merging method is sequential. After copying the first sublist, the second merge processing approximately 2n/k elements, the third merge processing approximately 3n/k elements and this continues until the final merge processing n elements.
Therefore:
$T(n, k) = n/k + 2n/k + ... + k = n/k * (1+k)*k/2 = n * (1+k) /2$
$= O(nk)$

At every recursion level, it would cost $O(nk+n) = O(nk)$ for splitting and merge_sorted_list.

We have $\log_k n$ recursion levels, therefore:
$O(nk)*O(\log_k n) = O(nk \log_k n)$

Worst case: k=n
$O(nk \log_k n) = O(n^2)$

Best case: k=2
$O(nk \log_k n) = O(n\log n)$

Word Count

This document contains 252 words.

12 Invariant Proofs

Write a full invariant proof and demonstrate understanding of the invariants required for proving results.

Outcome	Description
TLO1	Understand how to write a formal proof for your code and algorithms.

Date	Author	Comment
2026/08/28 02:00	Yihao Yu	Working On It
2026/08/29 22:35	Yihao Yu	Ready for Feedback
2026/09/06 14:08	Gary Zheng	Hi Yihao, P7 looks good, happy to tick it off.
2026/09/06 14:08	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Invariant Proofs

Submitted By:
Yihao Yu
yyuu0140
2026/08/29 22:35

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Understand how to write a formal proof for your code and algorithms.	

August 29, 2026



P7 Report

Task 1

Divide by 3

```
int div_3(int arr[], int n) {
    // Counts the number of values in the array that are divisible by 3.
    // The length of arr is n.
    int count = 0;
    for (int i=0; i<n; i++) {
        // Your invariant should apply here!
        if (arr[i] % 3 == 0) {
            count ++;
        }
    }
    return count;
}
```

Invariant: At the beginning of each iteration, count is the number of values divisible among the first i elements in the array.

Sum of primes

```
int sum_of_primes(int n) {
    // Computes the sum of the first n primes.
    int total = 0;
    for (int i=1; i<n; i++) {
        // Your invariant should apply here!
        if (is_prime(i)) { // You may assume this function exists and
works.
            total = total + i;
        }
    }
    return total;
}
```

Invariant: At the beginning of each iteration, total is the number of all prime numbers less than i.

Shaker Sorting

```
void shaker_sort(int arr[], int n) {
    // Sorts the list of arr.
    // The length of arr is `n`.
    int i = 0;
    int j = n - 1;
    while (i < j) {
        // Your invariant should apply here!
        // Selects the index of the smallest item from arr[i:n]
        int small = smallest_item_index(arr, n, i);
        swap(arr[i], arr[small]);
        // Selects the index of the largest item from arr[0:j+1]
```

```
        int large = largest_item_index(arr, n, j);
        swap(arr[j], arr[large]);
        i++;
        j--;
    }
}
```

Invariant: At the beginning of each loop iteration, the first i elements are the i smallest elements in their correct sorted positions, while the elements after index j are the n-1-j largest elements in their correct sorted positions.

Task 2

Invariant Proof for function div_3: At the beginning of each iteration, count is the number of values divisible by 3 among the first i elements of the array.

Initialisation: The first time the invariant is reached, i = 0 and count = 0.

Since none of the array elements have processed yet, the first 0 elements contain 0 values divisible by 3. Therefore, count correctly stores the number of values divisible by 3 among the first i = 0 elements.

Thus, the invariant holds before the first iteration.

Maintenance: Let k be an integer such that 0<= k <n. Suppose the invariant holds for i = k. That is count currently holds the number of values divisible by 3 among the first k elements of arr.
Then, we execute line 7 – one of two scenarios occurs:

Case1: arr[k] is divisible by 3
Then arr[k] % 3 == 0 is true, so the count is incremented by 1. Since count previously stored the number of divisible values among the first k elements, it now stores the number of divisible values among the first k+1 elements.
Case2: arr[k] is not divisible by 3
Then arr[k]% 3 == 0 is false, so the count is not changed. Since arr[k] does not add another value divisible by 3, count still correctly stores the number of divisible values among the first k+1 elements.

In either case, count currently holds the number of values divisible by 3 among the first k+1 elements.

Then, we execute the loop iteration code i++, in other words, we set i =k+1.

Then, the invariant line is researched again. We know that:
count currently holds the number of values divisible by 3 among the first k+1 elements.
i = k+1

So we have: the count currently holds the number of values divisible by 3 among the first k+1 = i elements

And so the invariant holds.

Termination

At the final occurrence of the invariant, before loop exit, we have i=n. From our maintained invariant, this implies count currently holds the number of values divisible by 3 among the first n elements. In other word, count holds the whole number of values divisible by 3 among the whole elements. As such, div_3 computes the correct value.

13 Batched Queries

Implement a Post Analyser supporting batched ingestion, removal, and threshold queries.

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/08/29 22:36	Yihao Yu	Working On It
2026/09/02 17:23	Yihao Yu	Ready for Feedback
2026/09/02 17:24	Gary Zheng	Tests complete: 4 / 4
2026/09/02 17:26	Yihao Yu	When I was designing the method remove_post which is inside the PostAnalyser, I just find that if I do not use BST - binary search tree, I could still use the binary search to find the index of post I want to delete, but when it comes to truely delete the corresponding elements in the array, I need to copy every elements except for the one I want to delete
2026/09/02 17:27	Yihao Yu	In that case, my time complexity is O(b log n + n)
2026/09/02 17:28	Yihao Yu	b log n is for the binary search, while the n is for copy the array
2026/09/02 17:29	Yihao Yu	So I just wanna know, if it is a must to use BST in this task since when I check the task planner in the ontrack, I find that the P8 is what I gonna do later
2026/09/02 17:29	Yihao Yu	Thanks for your reply in advance! I really appreciate your labour and work always!
2026/09/02 17:29	Yihao Yu	:+1::grin:
2026/09/07 16:14	Gary Zheng	Hi Yihao thank you for your C2 and apologise for getting back a bit late, the main p0oint of feedback is yes, you should use a BST for this task because as you recognised, when deleting from array you do need to shift everything.
2026/09/07 16:15	Gary Zheng	Feel free to talk to me in class if you want me to elaborate further for this task, but I think you have the correct idea.
2026/09/07 16:15	Gary Zheng	Fix and Resubmit
2026/09/08 17:03	Yihao Yu	Ready for Feedback
2026/09/08 17:04	Gary Zheng	Tests complete: 4 / 4
2026/09/13 14:36	Gary Zheng	HI Yihao, happy with your C2, please come see me for a discussion for this task.
2026/09/13 14:36	Gary Zheng	Discuss
2026/09/15 14:46	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Batched Queries

Submitted By:
Yihao Yu
yyuu0140
2026/09/08 17:03

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

September 8, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <string>

// ...
#include <codecvt>

#include "algorithms.h"
#include "data_structures.h"

using namespace std;

struct PostAnalyser {
    // ...
    LinkedList<Batch<Post>> batches;
    int batch_size;

    PostAnalyser(int n) {
        // ...
        batch_size = n;

    }

    void add_batch(Post* post_batch) {
        // ...
        merge_sort(post_batch, batch_size);
        Batch<Post> batch(post_batch, batch_size);
        batches.append(batch);
    }

    void remove_post(Post post) {
        // ...
        Node<Batch<Post>>* current_node = batches.first_node;
        while (current_node != nullptr)
        {
            current_node->item.delete_item(post);
            current_node = current_node->next;

        }
    }

    Post* least_popular_over_threshold(int threshold)
    {
        // ...
       Node<Batch<Post>>* current_node = batches.first_node;

        Post* result = nullptr;
        Post threshold_post;
        threshold_post.likes = threshold;
        threshold_post.post_name = "Threshold";

```

```
51
52          while (current_node != nullptr)
53          {
54              Post* post = current_node->item.least_over_threshold(threshold_post);
55
56              if (post != nullptr)
57              {
58                 if ( result == nullptr || post->likes < result->likes)
59                 {
60                     result = post;
61                 }
62              }
63              current_node = current_node->next;
64          }
65          return result;
66      }
67
68
69  };
70
71  int ontrack_test() {
72      int n;
73      cout << "Enter the batch size:" << endl;
74      cin >> n;
75      PostAnalyser analyser(n);
76
77      string menu = "1. Add a batch\n2. Remove a post\n3. Find least popular post over
        ↪  a threshold\n4. See the menu again\n5. Quit";
78      int command = 4;
79      while (command != 5) {
80          if (command == 1) {
81              Post* batch_post = new Post[n];
82              for (int i = 0; i < n; i++) {
83                  cout << "Enter likes for post " << i << ":" << endl;
84                  int likes;
85                  cin >> likes;
86
87                  cout << "Enter post name for post " << i << ":" << endl;
88                  string post_name;
89                  cin >> post_name;
90
91                  batch_post[i].likes = likes;
92                  batch_post[i].post_name = post_name;
93              }
94              analyser.add_batch(batch_post);
95              delete[] batch_post;
96          } else if (command == 2) {
97              cout << "Enter likes of the post to remove:" << endl;
98              int likes;
99              cin >> likes;
100
101             Post post;
102             post.likes = likes;
103             post.post_name = "To be deleted"; // The name doesn't matter for
                ↪  deletion, only likes are used for comparison
```

```
104             analyser.remove_post(post);
105         } else if (command == 3) {
106             cout << "Enter threshold:" << endl;
107             int threshold;
108             cin >> threshold;
109             Post* result = analyser.least_popular_over_threshold(threshold);
110             if (result == nullptr) {
111                 cout << "No such post exists" << endl;
112             } else {
113                 cout << "Post found with " << result->likes << " likes" << endl;
114             }
115         } else if (command == 4) {
116             cout << menu << endl;
117         }
118 
119         cout << "Enter command (1-5):" << endl;
120         cin >> command;
121         if (!cin) {
122             cout << "ERROR: could not read next command" << endl;
123             return 1;
124         }
125     }
126     return 0;
127 }
128 
129 
130 int main(int argc, char** argv) {
131     if (argc != 1) {
132         return ontrack_test();
133     }
134     // Include any tests you want to write in code here.
135     ontrack_test();
136 
137 
138     return 0;
139 }
```

```cpp
//
// Created by Yihao Yu on 2/9/2026.
//

#ifndef FIT2085_ALGORITHMS_H
#define FIT2085_ALGORITHMS_H


template<typename T>
void merge_sort(T arr[], int n)
{
    if (n <= 1)
    {
        return;
    }

    int left_size = n / 2;
    int right_size = n - left_size;

    T* left_half = new T[left_size];
    T* right_half = new T[right_size];

    for (int i = 0; i < n; i++)
    {
        if (i < left_size)
        {
            left_half[i] = arr[i];
        }else
        {
            right_half[i-left_size] = arr[i];
        }
    }

    merge_sort(left_half, left_size);
    merge_sort(right_half, right_size);

    int left_half_index = 0;
    int right_half_index = 0;
    int result_index = 0;

    while (left_half_index < left_size && right_half_index < right_size)
    {
        if (left_half[left_half_index] < right_half[right_half_index])
        {
            arr[result_index] = left_half[left_half_index];
            left_half_index++;
        }else
        {
            arr[result_index] = right_half[right_half_index];
            right_half_index++;
        }
        result_index++;
    }

```

```
55      while (left_half_index < left_size )
56      {
57          arr[result_index] = left_half[left_half_index];
58          result_index++;
59          left_half_index++;
60      }
61
62      while (right_half_index < right_size )
63      {
64          arr[result_index] = right_half[right_half_index];
65          result_index++;
66          right_half_index++;
67      }
68
69      delete[] left_half;
70      delete[] right_half;
71  }
72
73  template<typename T>
74  int binary_search(T arr[], int left, int right, T target)
75  {
76      if (left > right)
77      {
78          return -1;
79      }
80
81      int mid = (left+right)/2;
82
83      if (arr[mid] == target)
84      {
85          return mid;
86      }else if (arr[mid] > target)
87      {
88          return binary_search(arr, left, mid-1, target);
89      }else
90      {
91          return binary_search(arr, mid+1, right, target);
92      }
93  }
94
95  template<typename T>
96  int binary_search_threshold(T arr[], int left, int right, T target)
97  {
98      if (left > right)
99      {
100         return -1;
101     }
102
103     int mid = (left+right)/2;
104
105    if (arr[mid] > target)
106    {
107        if (mid == 0 || arr[mid-1] <= target )
108        {
```

```
109             return mid;
110         }
111 
112         return binary_search_threshold(arr, left, mid-1, target);
113     }
114 
115     return binary_search_threshold(arr, mid+1, right, target);
116 }
117 
118 #endif //FIT2085_ALGORITHMS_H
```

```cpp
//
// Created by Yihao Yu on 2/9/2026.
//


#ifndef FIT2085_DATA_STRUCTURES_H
#define FIT2085_DATA_STRUCTURES_H

#include <iostream>
#include <string>
using namespace std;

struct Post {
    int likes;
    string post_name;

    // ...
    Post()
    {
        this->likes = 0;
        this->post_name = "";
    }

    Post(int likes, string post_name)
    {
        this->likes = likes;
        this->post_name = post_name;
    }

    bool operator<(const Post& rhs) const
    {
        return this->likes < rhs.likes;
    }

    bool operator<=(const Post& rhs) const
    {
        return this->likes <= rhs.likes;
    }

    bool operator>(const Post& rhs) const
    {
        return this->likes > rhs.likes;
    }

    bool operator>=(const Post& rhs) const
    {
        return this->likes >= rhs.likes;
    }

    bool operator==(const Post& rhs) const
    {
        return this->likes == rhs.likes;
    }
};
```

```
55
56  /*
57  struct Batch
58  {
59      Post* posts;
60      int size;
61
62      Batch()
63      {
64          this->posts = nullptr;
65          this->size = 0;
66      }
67
68      Batch(Post* posts, int size)
69      {
70          this->posts = new Post[size];
71          this->size = size;
72
73          for (int i = 0; i < size; i++)
74          {
75              this->posts[i] = posts[i];
76          }
77      }
78
79      Batch(const Batch& batch)
80      {
81          this->size = batch.size;
82          this->posts = new Post[size];
83          for (int i = 0; i < size; i++)
84          {
85              this->posts[i] = batch.posts[i];
86          }
87      }
88
89      Batch& operator=(const Batch& batch)
90      {
91          if (this != &batch)
92          {
93              delete[] this->posts;
94
95              this->size = batch.size;
96              this->posts = new Post[size];
97              for (int i = 0; i < size; i++)
98              {
99                  this->posts[i] = batch.posts[i];
100             }
101
102             return *this;
103         }else
104         {
105             return *this;
106         }
107     }
108
```

```
    ~Batch()
    {
        delete[] posts;
    }

    void remove_post(int index)
    {
        int old_size = size;
        this->size = size-1;
        Post* new_posts = new Post[size];
        for (int i = 0; i < old_size; i++)
        {
            if (i < index)
            {
                new_posts[i] = this->posts[i];
            }
            else if (i > index)
            {
                new_posts[i-1] = this->posts[i];
            }
        }
        delete[] this->posts;
        this->posts = new_posts;
    }


};
*/

template<typename T>
struct Node
{
    T item;
    Node* next;
};

template<typename T>
struct LinkedList
{
    Node<T>* first_node = nullptr;
    Node<T>* last_node = nullptr;
    int size = 0;

    void append(const T& item)
    {
        Node<T>* new_node = new Node<T>;
        new_node->item = item;
        new_node->next = nullptr;

        if (first_node == nullptr)
        {
            first_node = new_node;
            last_node = new_node;
            size++;
```

```
163             return;
164         }else
165         {
166             last_node->next = new_node;
167             last_node = new_node;
168             size++;
169         }
170     }
171 
172     ~LinkedList()
173     {
174         Node<T>* current = first_node;
175 
176         while (current != nullptr)
177         {
178             Node<T>* next = current->next;
179             delete current;
180             current = next;
181         }
182     }
183 };
184 
185 template<typename T>
186 struct Batch_Node
187 {
188     T item;
189     Batch_Node<T>* left;
190     Batch_Node<T>* right;
191 };
192 
193 template<typename T>
194 struct Batch
195 {
196     Batch_Node<T>* root;
197     int size;
198 
199     Batch()
200     {
201         this->root = nullptr;
202         this->size = 0;
203     }
204 
205    Batch_Node<T>* build_balanced_tree(T* items, int left, int right)
206     {
207         if (left > right)
208         {
209             return nullptr;
210         }
211 
212         int mid = (left + right) / 2;
213 
214         Batch_Node<T>* node = new Batch_Node<T>;
215         node->item = items[mid];
216 
```

```
217         node->left = build_balanced_tree(items, left, mid - 1);
218         node->right = build_balanced_tree(items, mid + 1, right);
219 
220         return node;
221     }
222 
223     Batch(T* items, int size)
224     {
225         this->size = size;
226 
227         this->root = build_balanced_tree(items, 0, size - 1);
228     }
229 
230     Batch_Node<T>* copy_batch(Batch_Node<T>* node)
231     {
232         if (node == nullptr)
233         {
234             return nullptr;
235         }
236 
237         Batch_Node<T>* new_node = new Batch_Node<T>;
238         new_node->item = node->item;
239 
240         new_node->left = copy_batch(node->left);
241         new_node->right = copy_batch(node->right);
242 
243         return new_node;
244     }
245 
246     Batch(const Batch<T>& batch)
247     {
248         this->size = batch.size;
249         this->root = copy_batch(batch.root);
250     }
251 
252     void delete_batch(Batch_Node<T>* node)
253     {
254         if (node == nullptr)
255         {
256             return;
257         }
258 
259         delete_batch(node->right);
260         delete_batch(node->left);
261 
262         delete node;
263     }
264 
265     ~Batch()
266     {
267         delete_batch(root);
268     }
269 
270     Batch& operator=(const Batch& batch)
```

```
271     {
272         if (this != &batch)
273         {
274             delete_batch(this->root);
275 
276             this->size = batch.size;
277             this->root = copy_batch(batch.root);
278         }
279 
280         return *this;
281     }
282 
283     Batch_Node<T>* find_min(Batch_Node<T>* current_node)
284     {
285         while (current_node -> left != nullptr)
286         {
287             current_node = current_node -> left;
288         }
289 
290         return current_node;
291     }
292 
293     Batch_Node<T>* delete_helper(Batch_Node<T>* current_node, T item)
294     {
295         if (current_node == nullptr)
296         {
297             return nullptr;
298         }
299 
300         if (item < current_node->item)
301         {
302             current_node->left = delete_helper(current_node->left, item);
303 
304             return current_node;
305         }
306 
307         if (item > current_node->item)
308         {
309             current_node->right = delete_helper(current_node->right, item);
310 
311             return current_node;
312         }
313 
314         // Found target
315 
316         // 0 children
317         if (
318             current_node->left == nullptr &&
319             current_node->right == nullptr)
320         {
321             delete current_node;
322             return nullptr;
323         }
324 
```

```
325         // only right children
326         if (current_node -> left == nullptr)
327         {
328             Batch_Node<T>* replacement = current_node->right;
329             delete current_node;
330             return replacement;
331         }
332 
333         // only left children
334         if (current_node -> right == nullptr)
335         {
336             Batch_Node<T>* replacement = current_node->left;
337             delete current_node;
338             return replacement;
339         }
340 
341         // 2 children
342         Batch_Node<T>* successor = find_min(current_node->right);
343 
344         current_node -> item = successor->item;
345 
346         current_node -> right = delete_helper(current_node->right, successor->item);
347 
348         return current_node;
349     }
350 
351     void delete_item(T item)
352     {
353         root = delete_helper(root, item);
354     }
355 
356     T* least_over_threshold(T threshold)
357     {
358         Batch_Node<T>* current_node = root;
359         T* result = nullptr;
360 
361         while (current_node != nullptr)
362         {
363             if (current_node->item > threshold)
364             {
365                 result = &current_node -> item;
366 
367                 current_node = current_node -> left;
368             }else
369             {
370                 current_node = current_node -> right;
371             }
372         }
373 
374         return result;
375     }
376 
377 };
378 
```

```
379
380
381 #endif //FIT2085_DATA_STRUCTURES_H
```

C2 Report

Algorithm / Data Structure Explanation

I store the batches in a generic linked list because the number of batches is now not known in advance. Each batch stores the posts in binary search tree, ordered by their number of likes.

When a batch is added, generic merge sort is used to sort its post in ascending order of likes. This cost O(n log n). A BST is then constructed recursively by repeatedly selecting the middle element of each sorted range, taking O(n). Therefore, add_batch has an overall worst-case complexity of O(nlog n).

To remove a post, each batch BST is searched using its likes, a search and deletion takes O (log n) when the tree height is logarithmic, so checking up to up to b batches takes O (b log n)

For a threshold query, each BST is searched along one root-to-leaf path to find its smallest post above the threshold. Comparing the result from all b batches gives O(b log n).

Correctness Argument

Each batch is stored as a BST ordered by the number of likes. To find the least popular post above the threshold within one batch, the search starts at the root. If the current post has more likes than the threshold, it is a valid candidate, but a smaller valid post may exist in its left subtree. Therefore, the current post is stored as the result and the search continues left. If the current post has likes less than or equal to the threshold, neither it nor any post in its left subtree can satisfy the condition, so the search continues right.

When the search reaches nullptr, the stored candidate is therefore the smallest post in that batch whose likes are strictly greater than the threshold. PostAnalyser performs this search for every batch and keeps the candidate with the smallest number of likes. Consequently, after all batches have been examined, result points to the globally least-liked post above the threshold. If no batch contains a valid post, result remains nullptr.

Each balanced BST search follows one root-to-leaf path of height O(log n). Searching all b batches therefore takes O(b log n).

Testing Evidence

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/C2/C2task
Enter the batch size:
3
1. Add a batch
2. Remove a post
3. Find least popular post over a threshold
4. See the menu again
5. Quit
Enter command (1-5):
1
Enter likes for post 0:
2
Enter post name for post 0:
A
Enter likes for post 1:
1
Enter post name for post 1:
B
Enter likes for post 2:
3
Enter post name for post 2:
C
Enter command (1-5):
4
1. Add a batch
2. Remove a post
3. Find least popular post over a threshold
4. See the menu again
5. Quit
Enter command (1-5):
1
Enter likes for post 0:
4
Enter post name for post 0:
D
```

```
Enter likes for post 1:
5
Enter post name for post 1:
E
Enter likes for post 2:
6
Enter post name for post 2:
F
Enter command (1-5):
1
Enter likes for post 0:
10
Enter post name for post 0:
G
Enter likes for post 1:
11
Enter post name for post 1:
H
Enter likes for post 2:
12
Enter post name for post 2:
I
Enter command (1-5):
2
Enter likes of the post to remove:
10
Enter command (1-5):
4
```

```
1. Add a batch
2. Remove a post
3. Find least popular post over a threshold
4. See the menu again
5. Quit
Enter command (1-5):
3
Enter threshold:
9
Post found with 11 likes
Enter command (1-5):
3
Enter threshold:
10
Post found with 11 likes
Enter command (1-5):
3
Enter threshold:
0
Post found with 1 likes
Enter command (1-5):
2
Enter likes of the post to remove:
1
Enter command (1-5):
3
Enter threshold:
0
Post found with 2 likes
Enter command (1-5):
5
```

14 Supercharged BSTs

Store more in and do more with BST!

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/08/31 01:25	Yihao Yu	Working On It
2026/09/03 21:41	Yihao Yu	Not Started
2026/09/08 17:03	Yihao Yu	Working On It
2026/09/11 00:10	Yihao Yu	Ready for Feedback
2026/09/11 00:11	Gary Zheng	Tests complete: 6 / 6
2026/09/15 13:17	Gary Zheng	Hi Yihao, thank you for your D1, looks good, please come see me for a discussion.
2026/09/15 13:30	Gary Zheng	Discuss
2026/09/15 15:25	Gary Zheng	Discussed in class
2026/09/15 15:25	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Supercharged BSTs

Submitted By:
Yihao Yu
yyuu0140
2026/09/11 00:10

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

September 11, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <climits>

using namespace std;

struct RangeAnswer {
    int min_value;
    int max_value;
};

// Enable debugging the RangeAnswer object.
// This way, you can do cerr << rangeAnswer and it will print the min/max values rather than erroring out.
ostream & operator << (ostream & outs, const RangeAnswer & vs) {
    return outs << "[" << vs.min_value << ", " << vs.max_value << "]";
}

struct Node {
    int value;
    Node* left = nullptr;
    Node* right = nullptr;

    bool has_left() {
        // if left is something other than null, that means there is a child
        return left != nullptr;
    }

    bool has_right() {
        return right != nullptr;
    }

    bool is_leaf() {
        // If it doesn't have a left child and it doesn't have a right child, it
        // must be a leaf node.
        return !has_left() && !has_right();
    }

    int node_count() {
        // TASK 1
        int count = 1;

        if (has_left())
        {
            count += left->node_count();
        }

        if (has_right())
        {
            count += right->node_count();
        }
```

```
50
51          return count;
52
53          /*
54           * int left_count = (left == nullptr) ? 0 : left->node_count()
55           * int right_count = (right == nullptr) ? 0 : right->node_count()
56           *
57           * return 1 + left_count + right_count
58           */
59
60
61      }
62
63      RangeAnswer value_range() {
64          // TASK 1
65          RangeAnswer result = {this->value, this->value};
66
67          if (is_leaf())
68          {
69              return result;
70          }
71
72          // only left child
73          if (has_left() && !has_right())
74          {
75              result = this->left->value_range();
76              if (this->value < result.min_value)
77              {
78                  result.min_value = this->value;
79              }
80              if (this->value > result.max_value)
81              {
82                  result.max_value = this->value;
83              }
84          }
85
86          // only right child
87          if (has_right() && !has_left())
88          {
89              result = this->right->value_range();
90              if (this->value < result.min_value)
91              {
92                  result.min_value = this->value;
93              }
94              if (this->value > result.max_value)
95              {
96                  result.max_value = this->value;
97              }
98          }
99
100         // both has right child and right child
101         if (has_left() && has_right())
102         {
103             RangeAnswer left_result = this->left->value_range();
```

```
104             RangeAnswer right_result = this->right->value_range();
105 
106             result = {
107                 (left_result.min_value < right_result.min_value) ?
                    ↪  left_result.min_value : right_result.min_value ,
108                 (left_result.max_value > right_result.max_value) ?
                    ↪  left_result.max_value : right_result.max_value
109             };
110 
111             if (this->value < result.min_value)
112             {
113                 result.min_value = this->value;
114             }
115             if (this->value > result.max_value)
116             {
117                 result.max_value = this->value;
118             }
119         }
120 
121         return result;
122 
123 
124     }
125 
126     // TASK 2 - Maintain these values correctly through insertion
127     RangeAnswer stored_value_range = {INT_MAX, INT_MIN};
128     int stored_node_count = 0;
129 
130 };
131 
132 struct BinarySearchTree {
133     Node* root = nullptr;
134 
135     /// Return the node in the tree with this value. If there is no such node,
        ↪  return nullptr.
136     Node* search(int value) {
137         return _search_helper(root, value);
138     }
139 
140     Node* _search_helper(Node* current_node, int value) {
141         // If we are outside the tree, that means the value can't be found
142         if (current_node == nullptr) return nullptr;
143         // If current_node node has the value, return it
144         if (current_node->value == value) return current_node;
145 
146         // Otherwise, check which child we should go to.
147         if (value > current_node->value) {
148             return _search_helper(current_node->right, value);
149         } else {
150             return _search_helper(current_node->left, value);
151         }
152     }
153 
154     bool insert(int value) {
```

```
155         // First, if the tree is empty, just make this the root node
156         if (root == nullptr) {
157             root = new Node;
158             root->value = value;
159 
160             root->stored_node_count = 1;
161             root->stored_value_range = {value,value};
162 
163             // The item wasn't found, so we return true (indicating an insert
                ↪  happened)
164             return true;
165         }
166         // Again, I need a helper function to handle my recursion input
167         return _insert_helper(root, value);
168     }
169 
170     bool _insert_helper(Node* current_node, int value) {
171         // If this node has the key, return as the key is already in the tree.
172         // Tell the caller that the key is not a new node.
173         // We are defining that if the function returns false, that means the item
            ↪  was already in the tree
174         if (current_node->value == value) return false;
175 
176         // Check which direction we should go
177         if (value > current_node->value) {
178             // We should go to the right
179             // If there is no right child, that means this is exactly where
180             // the new node should be inserted
181             if (current_node->right == nullptr) {
182                 current_node->right = new Node;
183                 current_node->right->value = value;
184 
185                 current_node->right->stored_node_count = 1;
186                 current_node->right->stored_value_range = {value,value};
187 
188                 current_node->stored_node_count += 1;
189 
190                 if (value < current_node->stored_value_range.min_value)
191                 {
192                     current_node->stored_value_range.min_value = value;
193                 }
194 
195                 if (value > current_node->stored_value_range.max_value)
196                 {
197                     current_node->stored_value_range.max_value = value;
198                 }
199 
200                 // Item was unique and an insertion happened, so we return true.
201                 return true;
202             }
203             // Otherwise, if there is a right child, continue there
204             //return _insert_helper(current_node->right, value);
205             bool inserted = _insert_helper(current_node->right, value);
206             if (inserted)
```

```
207             {
208                 current_node->stored_node_count += 1;
209                 if (value < current_node->stored_value_range.min_value)
210                 {
211                     current_node->stored_value_range.min_value = value;
212                 }
213                 if (value > current_node->stored_value_range.max_value)
214                 {
215                     current_node->stored_value_range.max_value = value;
216                 }
217             }
218 
219             return inserted;
220 
221 
222         } else {
223             // We should go to the left
224             // If there is no left child, that means this is exactly where
225             // the new node should be inserted
226             if (current_node->left == nullptr) {
227                 current_node->left = new Node;
228                 current_node->left->value = value;
229 
230                 current_node->left->stored_node_count = 1;
231                 current_node->left->stored_value_range = {value,value};
232 
233                 current_node->stored_node_count += 1;
234 
235                 if ( value < current_node->stored_value_range.min_value )
236                 {
237                     current_node->stored_value_range.min_value = value;
238                 }
239 
240                 if (value > current_node->stored_value_range.max_value)
241                 {
242                     current_node->stored_value_range.max_value = value;
243                 }
244 
245                 return true;
246             }
247             // Otherwise, if there is a left child, continue there
248             //return _insert_helper(current_node->left, value);
249             bool inserted = _insert_helper(current_node->left, value);
250             if (inserted)
251             {
252                 current_node->stored_node_count += 1;
253 
254                 if (value < current_node->stored_value_range.min_value)
255                 {
256                     current_node->stored_value_range.min_value = value;
257                 }
258                 if (value > current_node->stored_value_range.max_value)
259                 {
260                     current_node->stored_value_range.max_value = value;
```

```
                }
            }

            return inserted;
        }
    }


    int count_lt(int x)
    {
        // TASK 3 - How many nodes have a value less than x?
        // Your answer should have a complexity of O(h), where h is the height of
        ↪  the tree, and should depend on your implementation of task 2.
        Node* current_node = root;
        int count = 0;

        while (current_node != nullptr)
        {
            if (current_node->value < x)
            {
                count++;

                if (current_node->left != nullptr)
                {
                    count += current_node->left->stored_node_count;
                }

                current_node = current_node->right;
            }
            else
            {
                current_node = current_node->left;
            }
        }
            return count;
    }

    int count_range(int x, int y) {
        // TASK 4 - Count of nodes with value v satisfying x <= v <= y
        // Your answer should have a complexity of O(h), where h is the height of
        ↪  the tree
        Node* current_node = root;
        int count_less_than = 0;

        while (current_node != nullptr)
        {
            if (current_node->value <= y)
            {
                count_less_than++;

                if (current_node->left != nullptr)
                {
                    count_less_than += current_node->left->stored_node_count;
                }
```

```
313
314                 current_node = current_node->right;
315             }
316             else
317             {
318                 current_node = current_node->left;
319             }
320         }
321
322         current_node = root;
323         int count_greater_than = 0;
324
325         while (current_node != nullptr)
326         {
327             if (current_node->value >= x)
328             {
329                 count_greater_than++;
330
331                 if (current_node->right != nullptr)
332                 {
333                     count_greater_than += current_node->right->stored_node_count;
334                 }
335
336                 current_node = current_node->left;
337             }
338             else
339             {
340                 current_node = current_node->right;
341             }
342         }
343
344         return count_less_than+count_greater_than - root->stored_node_count;
345     }
346 };
347
348 int main(int argc, char** argv) {
349     if (argc == 1) {
350
351         BinarySearchTree bst;
352         bst.insert(50);
353         bst.insert(30);
354         bst.insert(70);
355         bst.insert(20);
356         bst.insert(40);
357         bst.insert(60);
358         bst.insert(80);
359
360         cout << bst.search(40)->value << endl; // Should print 40
361
362         try {
363             bst.insert(40); // Should throw an error
364         } catch (const char* msg) {
365             cout << msg << endl; // Should print "Value already present in BST"
366         }
```

```
367
368         cout << "=== TASK 1 ===" << endl;
369         cout << "Node Count: " << bst.root->node_count() << ", Range: " <<
            ↪  bst.root->value_range() << endl;
370         cout << "Left Count: " << bst.root->left->node_count() << ", Range: " <<
            ↪  bst.root->left->value_range() << endl;
371         cout << "Right Count: " << bst.root->right->node_count() << ", Range: " <<
            ↪  bst.root->right->value_range() << endl;
372
373         cout << "=== TASK 2 ===" << endl;
374         cout << "Node Count: " << bst.root->stored_node_count << ", Range: " <<
            ↪  bst.root->stored_value_range << endl;
375         cout << "Left Count: " << bst.root->left->stored_node_count << ", Range: "
            ↪  << bst.root->left->stored_value_range << endl;
376         cout << "Right Count: " << bst.root->right->stored_node_count << ", Range:
            ↪  " << bst.root->right->stored_value_range << endl;
377
378         cout << "=== TASK 3 ===" << endl;
379         cout << "Amount Less Than 50: " << bst.count_lt(50) << endl;
380         cout << "Amount Less Than 35: " << bst.count_lt(35) << endl;
381         cout << "Amount Less Than 65: " << bst.count_lt(65) << endl;
382
383         cout << "=== TASK 4 ===" << endl;
384         cout << "Amount between 0 and 50: " << bst.count_range(0, 50) << endl;
385         cout << "Amount between 25 and 65: " << bst.count_range(25, 65) << endl;
386         cout << "Amount between 70 and 70: " << bst.count_range(70, 70) << endl;
387         cout << "Amount between 30 and 70: " << bst.count_range(30, 70) << endl;
388
389         return 0;
390     } else {
391         // Ontrack test cases DO NOT MODIFY
392         int command = 0;
393         BinarySearchTree bst;
394         Node* context_node = nullptr;
395         string menu = "0. Show this menu\n1. Insert a value into bst\n2. Search for
            ↪  a node\n3. Call node_count\n4. Return stored_node_count\n5. Call
            ↪  value_range
396 \n6. Return stored_value_range\n7. Call count_lt\n8. Call count_range\n9. Exit";
397         int value;
398         while (command != 9) {
399             try {
400                 if (command == 0) {
401                     cout << menu << endl;
402                 } else if (command == 1) {
403                     cout << "Enter value to insert: " << endl;
404                     cin >> value;
405                     bst.insert(value);
406                 } else if (command == 2) {
407                     cout << "Enter value to search for: "<< endl;
408                     cin >> value;
409                     context_node = bst.search(value);
410                     if (context_node) {
411                         cout << "Found node with " << context_node->value << endl;
412                     } else {
```

```
413                         cout << "Could not find node with value " << value << endl;
414                     }
415                 } else if (command == 3) {
416                     if (context_node) {
417                         cout << "Node count called as " <<
                            ↪  context_node->node_count() << endl;
418                     } else throw "ERROR: Context node is null";
419                 } else if (command == 4) {
420                     if (context_node) {
421                         cout << "Node count stored as " <<
                            ↪  context_node->stored_node_count << endl;
422                     } else throw "ERROR: Context node is null";
423                 } else if (command == 5) {
424                     if (context_node) {
425                         cout << "Value range called as " <<
                            ↪  context_node->value_range() << endl;
426                     } else throw "ERROR: Context node is null";
427                 } else if (command == 6) {
428                     if (context_node) {
429                         cout << "Value range stored as " <<
                            ↪  context_node->stored_value_range << endl;
430                     } else throw "ERROR: Context node is null";
431                 } else if (command == 7) {
432                     cout << "Enter upper bound: " << endl;
433                     cin >> value;
434                     cout << "Amount less than " << value << " is " <<
                        ↪  bst.count_lt(value) << endl;;
435                 } else if (command == 8) {
436                     cout << "Enter lower bound: " << endl;
437                     cin >> value;
438                     int ub;
439                     cout << "Enter upper bound: " << endl;
440                     cin >> ub;
441                     cout << "Number of nodes between " << value << " and " << ub <<
                        ↪  " is " << bst.count_range(value, ub) << endl;
442                 }
443 
444             } catch (const char* msg) {
445                 cout << msg << endl;
446             }
447             cout << "Enter command (0-9): " << endl;
448             if (!(cin >> command)) {
449                 cout << "ERROR: Could read next command" << endl;
450                 return 1;
451             };
452         }
453         return 0;
454     }
455 }
```

D1 Report

Reasoning - What nodes are impacted by insertions

Worded Argument

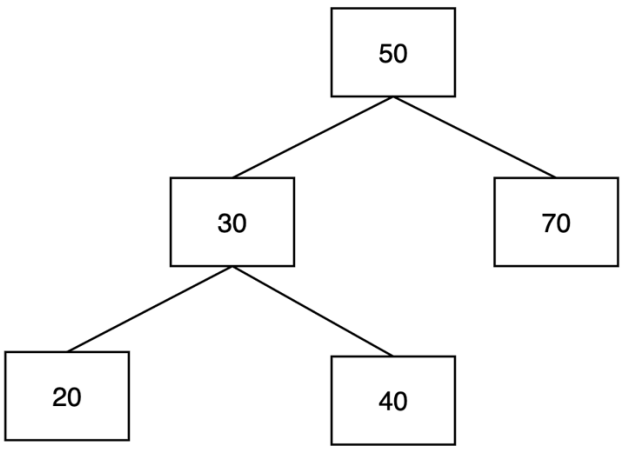
When a new node is inserted into a Binary Search Tree, the nodes that may return different values for node_count() and value_range() are the new node and its ancestors along the insertion path from the root.

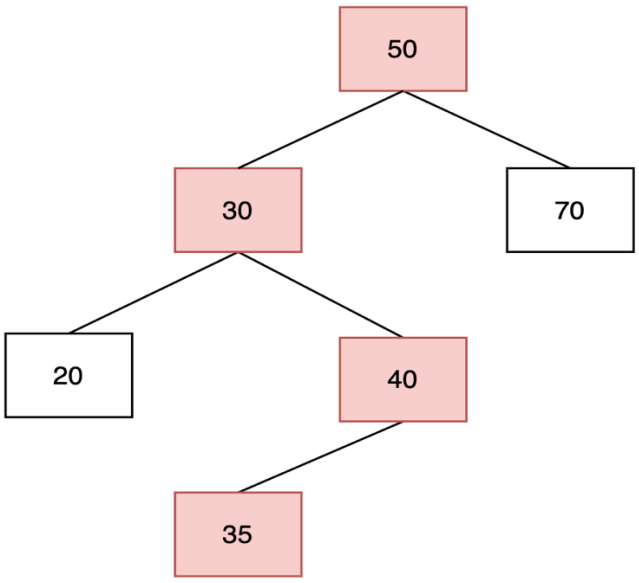
For example, when inserting 35 into the tree shown in the diagram, the insertion path is 50 → 30 → 40 → 35. The new node becomes part of the subtrees rooted at 40, 30, and 50. Therefore, the node count of each of these ancestor subtrees increases by one. Their value ranges may also change if the inserted value becomes a new minimum or maximum for the corresponding subtree.

Nodes that are not ancestors of the inserted node, such as 20 and 70 in the example, are unaffected. Their subtrees do not contain the newly inserted node, so neither their node count nor their value range changes.

Therefore, during insertion, the stored properties only need to be initialised for the new node and updated for its ancestors along the insertion path. Since the insertion path has a maximum length proportional to the height of the tree, these properties can be maintained without traversing unrelated subtrees.

Diagram

	Diagram
Before Insertion	

After Insertion	

15 Tree Structures

Determine the suitable Tree structure for the tasks and use them to solve the problems.

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/09/03 21:41	Yihao Yu	Working On It
2026/09/07 16:16	Yihao Yu	Ready for Feedback
2026/09/07 16:17	Gary Zheng	Tests complete: 0 / 4
2026/09/07 16:17	Gary Zheng	Fix and Resubmit
2026/09/07 16:17	Gary Zheng	**Automated comment**: Some tests did not pass for this submission. Please review the Overseer report, verify your output, and resubmit.
2026/09/07 16:53	Yihao Yu	Ready for Feedback
2026/09/07 16:55	Gary Zheng	Tests complete: 4 / 4
2026/09/08 13:11	Gary Zheng	Hi Yihao, thank you for your P8, looks good, just need to do a discussion to check your understanding then happy to tick it off.
2026/09/08 13:11	Gary Zheng	Discuss
2026/09/08 14:48	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Tree Structures

Submitted By:
Yihao Yu
yyuu0140
2026/09/07 16:53

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

September 7, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <string>
#include <cmath>

#include "structures.h"

using namespace std;

// One BST and one Max Heap already created for you in the global scope.
// These can be used in all of the functions below.
BinarySearchTree bst;
MaxHeap heap;

void add_post(Post* post) {
    // ...
    heap.add(post);
}

Node* search_node(Node* node, string username)
{
    if (node == nullptr || node->key == username)
    {
        return node;
    }

    if (username < node->key)
    {
        return search_node(node->left, username);
    }

    return search_node(node->right, username);
}

void print_trending(int k) {
    // ...
    // This is the print format expected for each trending post.
    // Feel free to use it in your code when needed:
    // cout << "Trending post by " << username << " with " << likes << " likes." << endl;
    for (int i = 0; i < k; i++)
    {
        Post* post = heap.extract_max();

        if (post == nullptr)
        {
            return;
        }

        cout << "Trending post by " << post->username << " with " << post->likes << " likes." << endl;
```

```
49
50          Post* current_best = bst.search(post->username);
51
52          if (current_best == nullptr)
53          {
54              bst.insert(post->username, post);
55          }
56          else if (current_best->likes < post->likes)
57          {
58              Node* node = search_node(bst.root, post->username);
59              node->post = post;
60          }
61      }
62  }
63
64  Post* best_trending_of_user(string username) {
65      // ...
66      return bst.search(username);
67  }
68
69  int get_height (Node* node)
70  {
71      if (node == nullptr)
72      {
73          return -1;
74      }
75
76      int left_height = get_height(node->left);
77      int right_height = get_height(node->right);
78
79      return 1 + max(left_height, right_height);
80  }
81
82  bool is_system_efficient(BinarySearchTree* system_bst) {
83      // ...
84      int n = system_bst->size;
85
86      if (n == 0)
87      {
88          return true;
89      }
90
91      int h = get_height(system_bst->root);
92
93      return pow(2,h) <= 5*n;
94
95  }
96
97  void test_is_system_efficient() {
98      /* SAMPLE TEST */
99      // Create a new BST
100     BinarySearchTree test_bst;
101     // Add a bunch of keys that will form a line, to make sure the tree is very
        ↪ unbalanced
```

```
102     test_bst.insert("a", nullptr);
103     test_bst.insert("b", nullptr);
104     test_bst.insert("c", nullptr);
105     test_bst.insert("d", nullptr);
106     test_bst.insert("e", nullptr);
107     test_bst.insert("f", nullptr);
108
109     // Call the function, see what it says for this BST
110     bool function_response = is_system_efficient(&test_bst);
111     // If it says it's not efficient, then the test has passed.
112     if (!function_response) cout << "Test 1 for system_efficient PASSED." << endl;
113     // Otherwise, if the code thinks this tree is efficient, it has failed.
114     else cout << "Test 1 for system_efficient FAILED." << endl;
115
116     /* YOUR TESTS */
117     // ...
118     BinarySearchTree balanced_bst;
119     balanced_bst.insert("d", nullptr);
120     balanced_bst.insert("b", nullptr);
121     balanced_bst.insert("f", nullptr);
122     balanced_bst.insert("a", nullptr);
123     balanced_bst.insert("c", nullptr);
124     balanced_bst.insert("e", nullptr);
125     balanced_bst.insert("g", nullptr);
126
127     bool function_response_2 = is_system_efficient(&balanced_bst);
128     if (function_response_2 ) cout << "Test 2 for system_efficient PASSED." << endl;
129     else cout << "Test 2 for system_efficient FAILED." << endl;
130
131     BinarySearchTree unbalanced_bst;
132     unbalanced_bst.insert("g", nullptr);
133     unbalanced_bst.insert("f", nullptr);
134     unbalanced_bst.insert("e", nullptr);
135     unbalanced_bst.insert("d", nullptr);
136     unbalanced_bst.insert("c", nullptr);
137     unbalanced_bst.insert("b", nullptr);
138     unbalanced_bst.insert("a", nullptr);
139
140     bool function_response_3 = is_system_efficient(&unbalanced_bst);
141     if (!function_response_3) cout << "Test 3 for system_efficient PASSED." << endl;
142     else cout << "Test 3 for system_efficient FAILED." << endl;
143 }
144
145 int ontrack_tests() {
146     string menu = "1. Add a post\n2. Get the next trending list\n3. Check user's
      ↪  trending post\n4. Test if a tree is efficient\n5. See the menu again\n6.
      ↪  Quit";
147
148     cout << menu << endl;
149     int command = 0;
150     cout << "Enter command (1-6):" << endl;
151     cin >> command;
152
153     while (command != 6) {
```

```
154         if (command == 1) {
155             cout << "Enter the username of the poster:" << endl;
156             string username;
157             cin >> username;
158 
159             cout << "Enter the number of likes:" << endl;
160             int likes;
161             cin >> likes;
162 
163             Post* post = new Post;
164             post->username = username;
165             post->posted_at = time(0);
166             post->likes = likes;
167             post->comments = 0;
168 
169             add_post(post);
170 
171         } else if (command == 2) {
172             cout << "Enter how many posts to read from the trending list:" << endl;
173             int k;
174             cin >> k;
175 
176             print_trending(k);
177         } else if (command == 3) {
178             cout << "Enter the username:" << endl;
179             string username;
180             cin >> username;
181 
182             Post* post = best_trending_of_user(username);
183             if (post == nullptr) {
184                 cout << "User does not have any trending posts" << endl;
185             } else {
186                 cout << "User's best trending post has " << post->likes << "
                    ↪  likes." << endl;
187             }
188         } else if (command == 4) {
189             int n;
190             cout << "Enter number of nodes" << endl;
191             cin >> n;
192             BinarySearchTree temp = BinarySearchTree();
193             cout << "Enter node keys:" << endl;
194             for (int i = 0; i < n; i++) {
195                 string key;
196                 cin >> key;
197                 temp.insert(key, nullptr);
198             }
199             if (is_system_efficient(&temp)) {
200                 cout << "Tree is efficient" << endl;
201             } else {
202                 cout << "Tree is not efficient" << endl;
203             }
204         } else if (command == 5) {
205             cout << menu << endl;
206         } else {
```

```
207             cout << "Unknown command." << endl;
208         }
209
210         cout << "Enter command (1-6):" << endl;
211         cin >> command;
212         if (!cin) {
213             cerr << "ERROR: could not read next command" << endl;
214             return 1;
215         }
216     }
217
218     return 0;
219 }
220
221 int main(int argc, char** argv) {
222
223     if (argc == 2) {
224         return ontrack_tests();
225     }
226
227     // Once the commands above are done, the tests for system efficiency will run
        ↪ here.
228     test_is_system_efficient();
229
230     return 0;
231 }
```

P8 Report

Time Complexity

Let n be the number of posts currently stored in the Max Heap, k be the number of trending post requested, and h be the number of BST's height.

For each post display, print_trending calls extract_max on the Max Heap. Because the heap is a complete binary tree, its height is $\log_2 n$, so extract_max takes O(log n).

The function also searches the BST for the username and may insert a new entry or updating an existing one. Therefore, these BST operations take O(h).

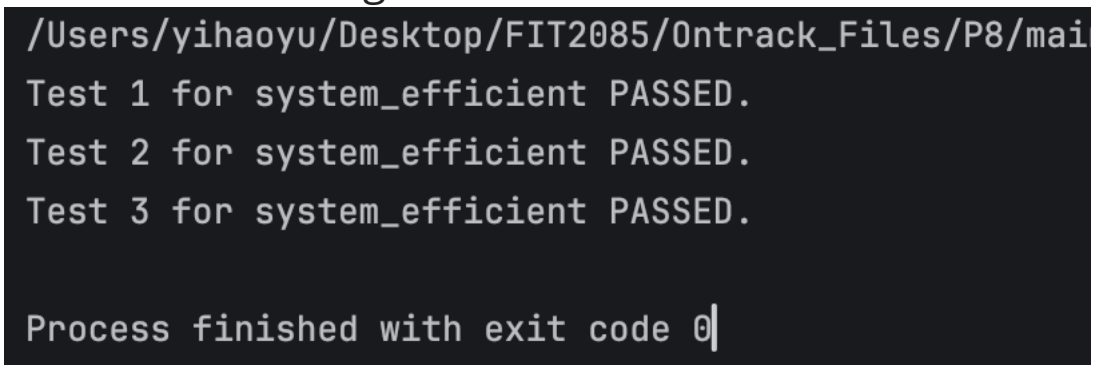
Above all, processing one post takes O(log n + h), and processing up to k posts takes:

O (k (log n + h))

For the best case, h = $\log_2 n$, the complexity is O (k log n).

For the worst case, h = n – 1, the complexity is O(kn).

Evidence of Testing



```
yihaoyu@dyn-118-138-78-77 P8 % ./main test
1. Add a post
2. Get the next trending list
3. Check user's trending post
4. Test if a tree is efficient
5. See the menu again
6. Quit
Enter command (1-6):
1
Enter the username of the poster:
A
Enter the number of likes:
1
Enter command (1-6):
1
Enter the username of the poster:
B
Enter the number of likes:
2
Enter command (1-6):
3
Enter the username:
A
User does not have any trending posts
Enter command (1-6):
5
1. Add a post
2. Get the next trending list
3. Check user's trending post
4. Test if a tree is efficient
5. See the menu again
6. Quit
Enter command (1-6):
1
Enter the username of the poster:
C
Enter the number of likes:
3
Enter command (1-6):
1
Enter the username of the poster:
D
Enter the number of likes:
4
```

```
Enter command (1-6):
1
Enter the username of the poster:
E
Enter the number of likes:
5
Enter command (1-6):
1
Enter the username of the poster:
F
Enter the number of likes:
6
Enter command (1-6):
1
Enter the username of the poster:
G
Enter the number of likes:
7
Enter command (1-6):
5
1. Add a post
2. Get the next trending list
3. Check user's trending post
4. Test if a tree is efficient
5. See the menu again
6. Quit
Enter command (1-6):
2
Enter how many posts to read from the trending list:
2
Trending Post byG with 7 likes.
Trending Post byF with 6 likes.
Enter command (1-6):
4
Enter number of nodes
1
Enter node keys:
1
Tree is efficient
Enter command (1-6):
3
Enter the username:
a
User does not have any trending posts
Enter command (1-6):
3
Enter the username:
G
```

```
User's best trending post has 7 likes.
Enter command (1-6):
3
Enter the username:
F^[
User does not have any trending posts
Enter command (1-6):
3
Enter the username:
E
User does not have any trending posts
Enter command (1-6):
3
Enter the username:
F
User's best trending post has 6 likes.
Enter command (1-6):
3
Enter the username:
G
User's best trending post has 7 likes.
```

```
 Enter command (1-6):
 1
 Enter the username of the poster:
 G
 Enter the number of likes:
 100
 Enter command (1-6):
 2
 Enter how many posts to read from the trending list:
 1
 Trending Post byG with 100 likes.
 Enter command (1-6):
 3
 Enter the username:
 G
 User's best trending post has 100 likes.
```

16 Monotonic Stack

Solve complex problems with a variation of stacks.

Outcome	Description
TLO1	Research new algorithms and data structures on your own, and understand how they can be used for solving problems.

Date	Author	Comment
2026/09/11 00:50	Yihao Yu	Working On It
2026/09/28 21:38	Yihao Yu	Ready for Feedback
2026/09/28 21:38	Gary Zheng	Tests complete: 6 / 6
2026/09/29 12:06	Gary Zheng	Hi Yihao, thank you for your HD1 submission, however there is one slight issue:One of the main points of this task is for you to learn about a new data structure, then implement it as its own type. We can technically treat a normal stack as a monotonic stack, but it's a better design if our type was specifically a monotonic stack and every push would automatically maintain its order, so we don't have to have that as an extra bit of work in the main program logic.
2026/09/29 12:06	Gary Zheng	Let me know if you have any questions
2026/09/29 12:06	Gary Zheng	Fix and Resubmit
2026/09/29 14:47	Yihao Yu	Ready for Feedback
2026/09/29 14:48	Gary Zheng	Tests complete: 6 / 6
2026/09/29 18:34	Gary Zheng	Hi Yihao, For your Monotonic stack push, it shouldn't look like this:"oid push(Block* block)33 {34 if (top == nullptr)35 {36 top = block;37 return;38 }39 block->next = top;40 top = block;41 }"
2026/09/29 18:34	Gary Zheng	Remember that when we push in MS Stack, we need to pop off all the elements that does not maintain stack order first?
2026/09/29 18:34	Gary Zheng	Fix and Resubmit
2026/09/29 18:35	Gary Zheng	Oh sorry I saw monotonic push my bad, can you correct the code to just have monotonic push?Because the datastructure itself is a monotonic stack, so you should not have 2 versions of push. If this is confusing to you I can explain in more details next week.
2026/09/29 18:35	Gary Zheng	Fix and Resubmit
2026/10/02 01:14	Yihao Yu	Ready for Feedback
2026/10/02 01:14	Yihao Yu	Thanks for your feedback! I have already rebuild my push method so that there is only one method existing in my struct right now
2026/10/02 01:15	Gary Zheng	Tests complete: 6 / 6
2026/10/03 13:55	Gary Zheng	Hi Yihao, there is still an issue with your HD1 MSStack push:All the push method should be is:Pop everything less than the current element being pushed, then push the element, that's it, nothing else.It should not apply the keycard rule here, all of that should go in solve.I'l give you a hint, you should also write a method called peek(), because right now your solve never looks at the stack, but the answer is the top index of the gate that prevent access.

2026/10/03 13:55	Gary Zheng	Fix and Resubmit
2026/10/04 23:19	Yihao Yu	Ready for Feedback
2026/10/04 23:29	Yihao Yu	Hi Gary, I have already fix my code that, the push method no longer apply the keycard rule currently. The rule of keycard is applied in the solve method right now. Thanks for your reply in advance, I really appreciate it!
2026/10/04 23:32	Gary Zheng	Tests complete: 6 / 6
2026/10/05 13:47	Gary Zheng	Hi Yihao, this looks good now, sorry i may have misread something before.Happy to set this to discuss.
2026/10/05 13:47	Gary Zheng	Discuss
2026/10/06 14:12	Gary Zheng	Discussed in class
2026/10/06 14:12	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Monotonic Stack

Submitted By:
Yihao Yu
yyuu0140
2026/10/04 23:19

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Research new algorithms and data structures on your own, and understand how they can be used for solving problems.	

October 4, 2026



```cpp
#include <iostream>

using namespace std;

struct Block
{
    int index;
    Block* next;

    int gate;
    int keycard;

    Block(){}

    Block(int index, int* gates, int* keycards)
    {
        this->index = index;
        this->gate = gates[index];
        this->keycard = keycards[index];
        this->next = nullptr;
    }
};

struct Monotonic_stack
{
    Block* top=nullptr;

    bool is_empty()
    {
        return top == nullptr;
    }

    int peek()
    {
        return top->index;
    }

    void pop()
    {
        if (top == nullptr)
        {
            return;
        }

        Block* new_top = top->next;
        delete top;
        top = new_top;
    }

    void push(Block* block)
    {
        while (!is_empty() && block->gate >= top->gate)
        {
            pop();
```

```
55          }
56
57          if (top == nullptr)
58          {
59              top = block;
60          }else
61          {
62              block->next = top;
63              top = block;
64          }
65
66      }
67
68      ~Monotonic_stack()
69      {
70          while (!is_empty())
71          {
72              this->pop();
73          }
74      }
75  };
76
77  int* solve(int n, int gates[], int keycards[]) {
78      // Solve the reachability problem described in the spec.
79      // n is the number of gates/keycards.
80      // gates is the clearance level of each gate, read from top to bottom
81      // keycards is the clearance level of each keycard, read from top to bottom
82      // You should return an n length array where the ith index tells us how many
        ↪  gates could be opened from the ith room (top to bottom).
83
84      // In the example in the spec, the input would be:
85      // n: 8
86      // gates: [5, 2, 3, 2, 2, 3, 1, 2]
87      // keycards: [1, 4, 1, 1, 3, 2, 0, 2]
88      // And your function should return:
89      // [0, 1, 0, 0, 4, 0, 0, 2]
90      int* result = new int[n];
91
92      Monotonic_stack* stack = new Monotonic_stack();
93
94      for (int i = 0; i < n; i++)
95      {
96          stack->push(new Block(i, gates,keycards));
97
98          while (!stack->is_empty() && keycards[i] >= gates[stack->peek()])
99          {
100             stack->pop();
101         }
102
103         if (stack->is_empty())
104         {
105             result[i] = i + 1;
106         }
107         else
```

```
108         {
109             result[i] = i - stack->peek();
110         }
111 
112     }
113 
114     delete stack;
115     return result;
116 }
117 
118 int ontrack_test() {
119     int n;
120     int* gates;
121     int* keycards;
122     cout << "Enter n: " << endl;
123     cin >> n;
124     gates = new int[n];
125     keycards = new int[n];
126     cout << "Enter gates: " << endl;
127     for (int i = 0; i < n; i++) {
128         cin >> gates[i];
129     }
130     cout << "Enter keycards: " << endl;
131     for (int i = 0; i < n; i++) {
132         cin >> keycards[i];
133     }
134     if (!cin) {
135         cout << "ERROR: could not read input data" << endl;
136         delete[] gates;
137         delete[] keycards;
138         return 1;
139     }
140 
141     int* result = solve(n, gates, keycards);
142     for (int i = 0; i < n; i++) {
143         cout << result[i] << " ";
144     }
145     cout << endl;
146     delete[] gates;
147     delete[] keycards;
148     delete[] result;
149     return 0;
150 }
151 
152 int main(int argc, char** argv) {
153     if (argc != 1) {
154         return ontrack_test();
155     }
156     int n = 8;
157     int* gates = new int[] {5, 2, 3, 2, 2, 3, 1, 2};
158     int* keycards =  new int[] {1, 4, 1, 1, 3, 2, 0, 2};
159     int* result = solve(n, gates, keycards);
160     for (int i = 0; i < n; i++) {
161         cout << result[i] << " ";
```

```
162     }
163     cout << endl;
164     delete[] gates;
165     delete[] keycards;
166     delete[] result;
167     return 0;
168 }
```

Monotonic Stack

Description of the Data Structure

A monotonic stack is a stack that maintains its elements in a monotonic order. My implementation uses a linked stack structure, where each Block stores the index, gate clearance, keycard clearance, and a pointer to the next block. The top pointer represents the top of the stack, so insertion and removal are performed only at the top. In this problem, the stack stores blocks representing gates that are still potential blockers. Before a new gate is pushed, gates with lower or equal clearance levels are removed if they are dominated by the new gate. Subsequently, keycards can also remove gates from the top when their clearance is sufficient to open them. Therefore, the stack only retains gates that may still prevent traversal from the current or a later room.

Why?

It is a monotonic stack because the clearance levels of the stored gates are maintained in strictly decreasing order from the bottom to the top. Before a new gate is pushed, all gates at the top with clearance less than or equal to the new gate are removed. This preserves the monotonic invariant and removes gates that can no longer be the nearest relevant blocker. A monotonic queue could similarly maintain elements in monotonic order, but it would require operations at both ends. For this problem, only the most recent relevant blocker needs to be examined and removed, so the LIFO behaviour of a stack is sufficient and simpler.

Complexity

A single monotonic-stack operation can take O(p) in the worst case because one insertion may cause many existing blocks to be popped. However, across p processed elements, each block is pushed onto the stack exactly once and can be popped at most once. Once a block is popped, it is permanently removed and cannot contribute to the cost of a later operation.Therefore, across all operations there are at most p pushes and p pops. The total number of stack operations is at most 2p, giving a time complexity of O(p).

Testing Evidence

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
8
Enter gates:
5
2
3
2
2
3
1
2
Enter keycards:
1
4
1
1
3
2
0
2
0 1 0 0 4 0 0 2
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
1
Enter gates:
5
Enter keycards:
5
1
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
1
Enter gates:
5
Enter keycards:
1
0
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
4
Enter gates:
5
4
3
2
Enter keycards:
0
0
0
0
0 0 0 0
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
4
Enter gates:
5
4
3
2
Enter keycards:
10
10
10
10
1 2 3 4
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
3
Enter gates:
8
5
2
Enter keycards:
1
6
3
0 1 2
```

17 List of Followers

Use our default hash table and implement a TipTop feature with it.

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/09/12 16:00	Yihao Yu	Working On It
2026/09/12 17:53	Yihao Yu	Ready for Feedback
2026/09/12 17:53	Gary Zheng	Tests complete: 1 / 4
2026/09/12 17:53	Gary Zheng	**Automated comment**: Some tests did not pass for this submission. Please review the Overseer report, verify your output, and resubmit.
2026/09/12 18:07	Yihao Yu	Ready for Feedback
2026/09/12 18:08	Gary Zheng	Tests complete: 1 / 4
2026/09/12 18:08	Gary Zheng	**Automated comment**: Some tests did not pass for this submission. Please review the Overseer report, verify your output, and resubmit.
2026/09/12 18:12	Yihao Yu	Ready for Feedback
2026/09/12 18:13	Gary Zheng	Tests complete: 1 / 4
2026/09/12 18:13	Gary Zheng	**Automated comment**: Some tests did not pass for this submission. Please review the Overseer report, verify your output, and resubmit.
2026/09/12 18:17	Yihao Yu	Ready for Feedback
2026/09/12 18:18	Gary Zheng	Tests complete: 4 / 4
2026/09/13 14:27	Gary Zheng	Hi Yihao, P9 looks good, please come see me for a discussion.
2026/09/13 14:27	Gary Zheng	Discuss
2026/09/15 14:37	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

List of Followers

Submitted By:
Yihao Yu
yyuu0140
2026/09/12 17:53

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

September 12, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <string>

#include "hash_table.h"

using namespace std;

// Record of the followers of one user
struct FollowersRecord {
    // The number of followers this record represents
    int n = 0;
    // The string array of those followers' usernames
    string* followers=nullptr;

    string serialise() {
        // ...
        if (n == 0)
        {
            return "";
        }

        string result = "";
        for (int i = 0; i < n-1; i++)
        {
            result += followers[i];
            result += "-";
        }
        result += followers[n-1];
        return result;
    }

    void deserialise(string record) {
        // ...
        delete[] followers;
        followers = nullptr;

        if (record == "")
        {
            n = 0;
            return;
        }

        n = 1;

        for (int i = 0; i < record.length(); i++)
        {
            if (record[i] == '-')
            {
                n++;
            }
```

```
        }

        followers = new string[n];
        int follower_index = 0;

        for (int i = 0; i < record.length(); i++)
        {
            if (record[i] == '-')
            {
                follower_index++;
                continue;
            }
            followers[follower_index] += record[i];
        }
    }
};

struct FollowersDatabase {
    HashTable ht;

    void add_follower(string username, string follower) {
        // ...
        string followers;

        try
        {
            followers = ht.search(username);
        }
        catch (string error)
        {
            ht.insert(username, follower);
            return;
        }


        FollowersRecord followers_record;
        followers_record.deserialise(followers);

        if (followers_record.n == 0)
        {
            ht.insert(username, follower);
            return;
        }

        for (int i = 0; i < followers_record.n; i++)
        {
            if (followers_record.followers[i] == follower)
            {
                return;
            }
        }

        string result = followers_record.serialise();
        result+= "-"+follower;
```

```
105         ht.insert(username, result);
106     }
107 
108     void delete_follower(string username, string follower) {
109         // ...
110         //string followers = ht.search(username);
111 
112         string followers;
113 
114         try
115         {
116             followers = ht.search(username);
117         }catch (string error)
118         {
119             cout<< "User does not have any followers";
120             return;
121         }
122 
123         FollowersRecord followers_record;
124         followers_record.deserialise(followers);
125 
126         if (followers_record.n == 0)
127         {
128             return;
129         }
130 
131         bool could_delete = false;
132 
133         for (int i = 0; i < followers_record.n; i++)
134         {
135             if (followers_record.followers[i] == follower)
136             {
137                 could_delete = true;
138                 break;
139             }
140         }
141 
142         if (!could_delete)
143         {
144             return;
145         }
146 
147 
148 
149         if (followers_record.n == 1)
150         {
151             ht.insert(username, "");
152             return;
153         }
154 
155         string result = "";
156         for (int i = 0; i < followers_record.n-1; i++)
157         {
158             if (followers_record.followers[i] == follower) continue;
```

```
159             result += followers_record.followers[i];
160             result += "-";
161         }
162 
163         if (followers_record.followers[followers_record.n - 1] != follower)
164         {
165             result += followers_record.followers[followers_record.n-1];
166         }else
167         {
168             result.pop_back();
169         }
170         ht.insert(username, result);
171     }
172 
173     void print_followers(string username) {
174         // ...
175         // At some point, you will need the follow print statement:
176         // cout << "User does not have any followers" << endl;
177         string followers;
178         try
179         {
180             followers = ht.search(username);
181         }catch (string error)
182         {
183             cout<< "User does not have any followers"<<endl;
184             return;
185         }
186         FollowersRecord followers_record;
187         followers_record.deserialise(followers);
188         if (followers_record.n == 0)
189         {
190             cout << "User does not have any followers" << endl;
191             return;
192         }
193         for (int i = 0; i < followers_record.n; i++)
194         {
195             cout << followers_record.followers[i] << endl;
196         }
197     }
198 };
199 
200 int ontrack_tests() {
201     FollowersDatabase db;
202 
203     string menu = "1. Add new follower for a user\n2. Remove a follower from a
    ↪  user\n3. Print all followers of a user\n4. See the menu\n5. Quit";
204 
205     cout << menu << endl;
206     int command = 0;
207     cout << "Enter command (1-5):" << endl;
208     cin >> command;
209 
210     while (command != 5) {
211         // The hash table methods throw when they are empty / non existant, so we
        ↪  wrap
```

```cpp
        // each action in a try/catch and report the error instead of crashing.
        try {
            if (command == 1) {
                cout << "Enter the username of the user who was followed:" << endl;
                string username;
                cin >> username;

                cout << "Enter the username of the follower:" << endl;
                string follower;
                cin >> follower;

                db.add_follower(username, follower);
            } else if (command == 2) {
                cout << "Enter the username of the user who was unfollowed:" <<
                ↪  endl;
                string username;
                cin >> username;

                cout << "Enter the username of the unfollower:" << endl;
                string unfollower;
                cin >> unfollower;

                db.delete_follower(username, unfollower);
            } else if (command == 3) {
                cout << "Enter a username to see their followers:" << endl;
                string username;
                cin >> username;

                db.print_followers(username);
            } else if (command == 4) {
                cout << menu << endl;
            } else {
                cout << "Unknown command." << endl;
            }
        } catch (const string error) {
            cout << "Error: " << error << endl;
        }

        cout << "Enter command (1-5):" << endl;
        cin >> command;
        if (!cin) {
            cerr << "ERROR: Could not read next command" << endl;
            return 1;
        }
    }
    return 0;
}

int main(int argc, char** argv) {
    if (argc == 2) {
        return ontrack_tests();
    }

    //test
```

```
265     FollowersDatabase db;
266     db.ht.insert("A", "");
267     db.ht.insert("B", "");
268 
269     //Test add_follower()
270     cout<< "Test add_follower()"<<endl;
271     cout<< "Add A1 for A:" << endl;
272     db.add_follower("A","A1");
273     db.print_followers("A");
274 
275     cout << "\nAdd B1, B2, B3 for B:"<<endl;
276     db.add_follower("B","B1");
277     db.add_follower("B","B2");
278     db.add_follower("B","B3");
279     db.print_followers("B");
280 
281     //Test delete_follower()
282     cout<< "\nTest delete_follower()"<<endl;
283     cout<< "Delete B2 for B:" << endl;
284     db.delete_follower("B","B2");
285     db.print_followers("B");
286 
287 
288     return 0;
289 }
```

P9 Report

Time Complexity

Let n be the number of followers stored for the specified user. Since usernames have a maximum length of 50 characters, so could treat the length of each username as a constant.

The deserialise operation scans the stored follower string and reconstructs the follower array, taking O(n) time. The function then searches up to n followers to determine whether the user has already existed in the array. If the follower is new, the serialise operation processes the follower list again, taking O(n) time.

Above all, ignoring the hash collisions, the whole complexity should be:

O(n).

However, if there are multiple keys points the same position in the hash table, we still need to take O(c) time, which c is the number of entries in that hash-table chain, to find the result needed in the worst case. Therefore, the worst case is O(n+c) and the best case is O(1) when the key is at the first node and the value of corresponding key is empty.

Evidence of Testing

```
FollowersDatabase db;
db.ht.insert(key: "A", value: "");
db.ht.insert(key: "B", value: "");

//Test add_follower()
cout<< "Test add_follower()"<<endl;
cout<< "Add A1 for A:" << endl;
db.add_follower(username: "A",follower: "A1");
db.print_followers(username: "A");

cout << "\nAdd B1, B2, B3 for B:"<<endl;
db.add_follower(username: "B",follower: "B1");
db.add_follower(username: "B",follower: "B2");
db.add_follower(username: "B",follower: "B3");
db.print_followers(username: "B");

//Test delete_follower()
cout<< "\nTest delete_follower()"<<endl;
cout<< "Delete B2 for B:" << endl;
db.delete_follower(username: "B",follower: "B2");
db.print_followers(username: "B");
```

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P9/main
Test add_follower()
Add A1 for A:
A1

Add B1, B2, B3 for B:
B1
B2
B3

Test delete_follower()
Delete B2 for B:
B1
B3
```

18 Infinite Hash Table

Handle hash table collisions recursively.

Outcome	Description
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.

Date	Author	Comment
2026/09/12 23:09	Yihao Yu	Working On It
2026/09/27 22:21	Yihao Yu	Ready for Feedback
2026/09/27 22:21	Gary Zheng	Tests complete: 2 / 4
2026/09/27 22:21	Gary Zheng	Fix and Resubmit
2026/09/27 22:21	Gary Zheng	**Automated comment**: Some tests did not pass for this submission. Please review the Overseer report, verify your output, and resubmit.
2026/09/27 23:10	Yihao Yu	Ready for Feedback
2026/09/27 23:16	Gary Zheng	Tests complete: 4 / 4
2026/10/03 14:49	Gary Zheng	Hi Yihao, good work for your D2, happy to tick this off.
2026/10/03 14:49	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Infinite Hash Table

Submitted By:
Yihao Yu
yyuu0140
2026/09/27 23:10

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Learn about the presented algorithms and data structures through solving problems and analysing them.	

September 27, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <string>
#include <vector>

using namespace std;

struct ContactBook;
struct ContactBookNode;

// You may assume that the User->nickname always matches the nickname it is inserted under.
struct User {
    string nickname;
    string display_name;
    int user_score;
};

struct ContactBookNode {
    string nickname;
    User* associated_user;
    // This will remain a nullptr if we don't see a collision.
    ContactBook* recursed_book = nullptr;

    // Define this after the ContactBook struct is defined.
    ~ContactBookNode();
};

struct ContactBook {
    const int table_size = 27;
    // Depth determines how the hash table functions
    int depth = 0;
    int size = 0;
    ContactBookNode** table = new ContactBookNode*[table_size];

    ContactBook() : ContactBook(0) {}
    ContactBook(int start_depth) : depth(start_depth)
    {
        for (int i = 0; i < table_size; i++)
        {
            table[i] = nullptr;
        }
    }
    ~ContactBook() {
        for (int i=0; i<table_size; i++) {
            delete table[i];
        }
        delete[] table;
    }

    int hash(const string& nickname) {
```

```
50          // Use the depth parameter!
51          if (depth >= nickname.length())
52          {
53              return 26;
54          }
55
56          return nickname[depth] - 'a';
57      }
58
59      void insert_user(const string& nickname, User* user_data) {
60          // Task 1
61          int index = hash(nickname);
62
63          // if table[index] is nullptr, we create a new Contact BookNode
64          if (table[index] == nullptr)
65          {
66              table[index] = new ContactBookNode;
67              table[index]->nickname = nickname;
68              table[index]->associated_user = user_data;
69
70              size++;
71              return;
72          }
73
74          // if table[index] is not nullptr and the recursed_book is not nullptr, we
            ↪ insert_user into the recursed_book
75          if (table[index]->recursed_book != nullptr)
76          {
77              table[index]->recursed_book->insert_user(nickname, user_data);
78
79              size++;
80              return;
81          }
82
83          // if table[index] is not nullptr and the recursed_book is nullptr, we
            ↪ firstly record the old_nickname and the old_user
84          string old_nickname = table[index]->nickname;
85          User* old_user = table[index]->associated_user;
86
87          // Subsequently, we create a recursed_book inside current contact book
88          table[index]->recursed_book = new ContactBook(depth + 1);
89
90          // we change the current contact book's name into "" and associated_user
            ↪ into nullptr
91          this->table[index]->nickname = "";
92          this->table[index]->associated_user = nullptr;
93
94          // At the last, we insert the old_nickname and old_user into corresponding
            ↪ recursed book
95          this->table[index]->recursed_book->insert_user(old_nickname, old_user);
96          this->table[index]->recursed_book->insert_user(nickname, user_data);
97
98          size++;
99
```

```
100     }
101
102     User* retrieve_user(const string& nickname) {
103         // Task 1
104         int index = hash(nickname);
105
106         // the table[index] is nullptr, we return nullptr
107         if (table[index] == nullptr)
108         {
109             return nullptr;
110         }
111
112         // if the table[index] has recursed_book, we retrieve user from its
            ↪  recursed_book
113         if (table[index]->recursed_book != nullptr)
114         {
115             return table[index]->recursed_book->retrieve_user(nickname);
116         }
117
118         // if the table[index] doesn't have recursed_book, we double check if the
            ↪  nickname of that is the same as nickname we want.
119         // If it is, we return the corresponding user
120         if (table[index]->nickname == nickname)
121         {
122             return table[index]->associated_user;
123         }
124
125         return nullptr;
126     }
127
128     // Used to get the only user from the ContactBook whose size is 1
129     User* get_only_user(string& nickname) {
130         for (int i = 0; i < table_size; i++)
131         {
132             if (table[i] == nullptr)
133             {
134                 continue;
135             }
136
137             if (table[i]->recursed_book != nullptr)
138             {
139                 return table[i]->recursed_book->get_only_user(nickname);
140             }
141
142             nickname = table[i]->nickname;
143             return table[i]->associated_user;
144         }
145
146         return nullptr;
147     }
148
149     void delete_user(const string& nickname) {
150         // Task 3
151         int index = hash(nickname);
```

```
152
153         // if table[index] is nullptr, just return
154         if (table[index] == nullptr)
155         {
156             return;
157         }
158
159         // if table[index] has recursed book, we execute delete_user inside the
            ↪  corresponding recursed book
160         if (table[index]->recursed_book != nullptr)
161         {
162             table[index]->recursed_book->delete_user(nickname);
163             size--;
164
165             // After the element is deleted, we check if the size of recursed_book
                ↪  inside is 1 or not.
166             // If the size is 1, we collapse the recursed book and change the the
                ↪  current table[index]'s nick name and
167             // associated user into remaining nickname and remaining user.
168             if (table[index]->recursed_book->size == 1)
169             {
170                 string remaining_nickname;
171                 User* remaining_user =
                    ↪  table[index]->recursed_book->get_only_user(remaining_nickname);
172
173                 delete table[index]->recursed_book;
174
175                 table[index]->recursed_book = nullptr;
176                 table[index]->nickname = remaining_nickname;
177                 table[index]->associated_user = remaining_user;
178             }
179
180             return;
181         }
182
183         if (table[index]->nickname == nickname)
184         {
185             delete table[index];
186             table[index] = nullptr;
187             size--;
188         }
189     }
190
191     bool search(const string& nickname) {
192         // Task 4
193         int index = hash(nickname);
194
195         if (table[index] == nullptr)
196         {
197             return false;
198         }
199
200         if (depth == nickname.length() - 1)
201         {
```

```
            return true;
        }

        if (table[index]->recursed_book != nullptr)
        {
            return table[index]->recursed_book->search(nickname);
        }

        string stored_nickname = table[index]->nickname;

        if (nickname.length() > stored_nickname.length())
        {
            return false;
        }

        for (int i = 0; i < nickname.length(); i++)
        {
            if (nickname[i] != stored_nickname[i])
            {
                return false;
            }
        }

        return true;
    }
};

ContactBookNode::~ContactBookNode() {
    // This needs to be defined after the ContactBook definition.
    delete recursed_book;
}

// Nicknames are only allowed to contain the lowercase characters a-z, so anything
// else would send hash() outside the bounds of the table. main() checks this before
// it hands anything to the ContactBook.
bool valid_nickname(const string& nickname) {
    if (nickname.empty()) {
        return false;
    }
    for (int i = 0; i < (int)nickname.length(); i++) {
        if (nickname[i] < 'a' || nickname[i] > 'z') {
            return false;
        }
    }
    return true;
}

int main() {
    ContactBook book;

    // The ContactBook only ever borrows a User*: insert_user stores the pointer,
    ↪ and
    // delete_user frees its own node but never the User behind it. So main() owns
    ↪ every
```

```
254     // User it allocates, and is responsible for deleting them again.
255     vector<User*> users;
256 
257     string menu = "1. Add a contact\n2. Retrieve a contact\n3. Remove a contact\n4.
        ↪  Search by prefix\n5. Show the number of contacts\n6. See the menu again\n7.
258 Quit";
259     int command = 6;
260     while (command != 7) {
261         if (command == 1) {
262             cout << "Enter the nickname:" << endl;
263             string nickname;
264             cin >> nickname;
265 
266             if (!valid_nickname(nickname)) {
267                 cout << "Nicknames must use the lowercase letters a-z only" << endl;
268             } else if (book.retrieve_user(nickname) != nullptr) {
269                 // Checked here so that insert_user is only ever called on a
                    ↪  nickname
270                 // we know is free, and never has to throw.
271                 cout << "The nickname '" << nickname << "' is already taken" <<
                    ↪  endl;
272             } else {
273                 cout << "Enter the display name:" << endl;
274                 string display_name;
275                 cin >> display_name;
276 
277                 cout << "Enter the user score:" << endl;
278                 int user_score;
279                 cin >> user_score;
280 
281                 User* user = new User;
282                 user->nickname = nickname;
283                 user->display_name = display_name;
284                 user->user_score = user_score;
285 
286                 book.insert_user(nickname, user);
287                 users.push_back(user); // main() keeps hold of it so it can free it
                    ↪  later
288                 cout << "Added " << display_name << " under the nickname '" <<
                    ↪  nickname << "'" << endl;
289             }
290         } else if (command == 2) {
291             cout << "Enter the nickname to retrieve:" << endl;
292             string nickname;
293             cin >> nickname;
294 
295             if (!valid_nickname(nickname)) {
296                 cout << "Nicknames must use the lowercase letters a-z only" << endl;
297             } else {
298                 User* user = book.retrieve_user(nickname);
299                 if (user == nullptr) {
300                     cout << "No contact is saved under '" << nickname << "'" <<
                        ↪  endl;
301                 } else {
```

```
302                     cout << "Nickname: " << user->nickname
303                          << " | Display name: " << user->display_name
304                          << " | Score: " << user->user_score << endl;
305                 }
306             }
307         } else if (command == 3) {
308             cout << "Enter the nickname to remove:" << endl;
309             string nickname;
310             cin >> nickname;
311 
312             User* user = valid_nickname(nickname) ? book.retrieve_user(nickname) :
                ↪ nullptr;
313             if (user == nullptr) {
314                 // Checked here so that delete_user is only ever called on a
                    ↪ nickname
315                 // we know is present, and never has to throw.
316                 cout << "No contact is saved under '" << nickname << "'" << endl;
317             } else {
318                 book.delete_user(nickname);
319 
320                 // The book has dropped its node, so now main() frees the User
                    ↪ itself.
321                 for (int i = 0; i < (int)users.size(); i++) {
322                     if (users[i] == user) {
323                         users.erase(users.begin() + i);
324                         break;
325                     }
326                 }
327                 delete user;
328                 cout << "Removed the contact '" << nickname << "'" << endl;
329             }
330         } else if (command == 4) {
331             cout << "Enter the prefix to search for:" << endl;
332             string prefix;
333             cin >> prefix;
334 
335             if (!valid_nickname(prefix)) {
336                 cout << "Prefixes must use the lowercase letters a-z only" << endl;
337             } else if (book.search(prefix)) {
338                 cout << "Yes - at least one contact has the prefix '" << prefix <<
                    ↪ "'" << endl;
339             } else {
340                 cout << "No contact has the prefix '" << prefix << "'" << endl;
341             }
342         } else if (command == 5) {
343             cout << "The contact book holds " << book.size << " contact(s)" << endl;
344         } else if (command == 6) {
345             cout << menu << endl;
346         }
347 
348         cout << "Enter command (1-7):" << endl;
349         if (!(cin >> command)) {
350             // End of input, or something that isn't a number - stop rather than
                ↪ loop forever.
```

```
351                 break;
352             }
353         }
354 
355         // Everything main() allocated, main() cleans up.
356         for (int i = 0; i < (int)users.size(); i++) {
357             delete users[i];
358         }
359         users.clear();
360 
361         return 0;
362 }
```

D2 Report

Data Structure Similarity

The ContactBook also represents a tree structure. The original ContactBook acts as the root, while each recursed_book created after a collision acts as a child at a deeper level. The depth of the tree corresponds to the character position used to hash the nickname. Nicknames that share the same characters remain on the same path, while different characters place them into different branches. Therefore, the recursive ContactBooks form a hierarchical tree in which nicknames are partially organised according to their characters.

Testing Evidence

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/D2/main
1. Add a contact
2. Retrieve a contact
3. Remove a contact
4. Search by prefix
5. Show the number of contacts
6. See the menu again
7. Quit
Enter command (1-7):
1
Enter the nickname:
alex
Enter the display name:
Alex
Enter the user score:
100
Added Alex under the nickname 'alex'
Enter command (1-7):
1
Enter the nickname:
alicia
Enter the display name:
Alicia
Enter the user score:
90
Added Alicia under the nickname 'alicia'
Enter command (1-7):
1
Enter the nickname:
bob
Enter the display name:
Bob
Enter the user score:
80
```

```
Added Bob under the nickname 'bob'
Enter command (1-7):
5
The contact book holds 3 contact(s)
Enter command (1-7):
2
Enter the nickname to retrieve:
alex
Nickname: alex | Display name: Alex | Score: 100
Enter command (1-7):
2
Enter the nickname to retrieve:
charlie
No contact is saved under 'charlie'
Enter command (1-7):
4
Enter the prefix to search for:
a
Yes - at least one contact has the prefix 'a'
Enter command (1-7):
4
Enter the prefix to search for:
al
Yes - at least one contact has the prefix 'al'
Enter command (1-7):
4
Enter the prefix to search for:
alex
Yes - at least one contact has the prefix 'alex'
Enter command (1-7):
4
Enter the prefix to search for:
ali
Yes - at least one contact has the prefix 'ali'
```

```
Enter command (1-7):
4
Enter the prefix to search for:
alice
No contact has the prefix 'alice'
Enter command (1-7):
4
Enter the prefix to search for:
bo
Yes - at least one contact has the prefix 'bo'
Enter command (1-7):
3
Enter the nickname to remove:
alicia
Removed the contact 'alicia'
Enter command (1-7):
5
The contact book holds 2 contact(s)
Enter command (1-7):
2
Enter the nickname to retrieve:
alicia
No contact is saved under 'alicia'
Enter command (1-7):
4
Enter the prefix to search for:
ali
No contact has the prefix 'ali'
```

Word Count

19 BST and Heap

What if we combined our tree structures?

Outcome	Description
TLO1	Explore how different data structures can be combined to create more powerful tools, and solve more complex problems.

Date	Author	Comment
2026/09/29 14:59	Yihao Yu	Working On It
2026/10/02 18:06	Yihao Yu	Ready for Feedback
2026/10/02 18:06	Gary Zheng	Tests complete: 1 / 2
2026/10/02 18:06	Gary Zheng	Fix and Resubmit
2026/10/02 18:06	Gary Zheng	**Automated comment**: Some tests did not pass for this submission. Please review the Overseer report, verify your output, and resubmit.
2026/10/02 18:16	Yihao Yu	Ready for Feedback
2026/10/02 18:18	Gary Zheng	Tests complete: 2 / 2
2026/10/03 15:45	Gary Zheng	Hi Yihao, thank you for your HD2, good work, I'm happy to set this to discuss.
2026/10/03 15:45	Gary Zheng	Discuss
2026/10/06 14:31	Gary Zheng	Complete
2026/10/06 14:31	Gary Zheng	Discussed in class

Monash

Fundamentals of Algorithms

OnTrack Submission

BST and Heap

Submitted By:
Yihao Yu
yyuu0140
2026/10/02 18:16

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Explore how different data structures can be combined to create more powerful tools, and solve more complex problems.	

October 2, 2026



This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```cpp
#include <iostream>
#include <random>

using namespace std;

template <typename T>
struct Node {
    int key;
    T value;
    int priority;
    Node<T>* left = nullptr;
    Node<T>* right = nullptr;
    Node<T>* parent = nullptr;

    inline static mt19937 gen{random_device{}()};

    Node(int k, T v, int p = -1) : key{k}, value{v} {
        priority = (p >= 0) ? p : uniform_int_distribution<int>(1, 10000)(gen);
    }

    bool has_left() {
        // if left is something other than null, that means there is a child
        return left != nullptr;
    }

    bool has_right() {
        return right != nullptr;
    }

    bool has_parent() {
        return parent != nullptr;
    }

    bool is_leaf() {
        // If it doesn't have a left child and it doesn't have a right child, it
        // must be a leaf node.
        return !has_left() && !has_right();
    }
};

template <typename T>
struct BinarySearchTree {
    Node<T>* root = nullptr;

    /// Return the node in the tree with this value. If there is no such node,
    ↪ return nullptr.
    Node<T>* search(int key) {
        return _search_helper(root, key);
    }

```

```
50      Node<T>* _search_helper(Node<T>* current_node, int key) {
51          // TODO
52          if (current_node == nullptr) return current_node;
53
54          if (current_node->key == key)
55          {
56              return current_node;
57          }
58
59          if (current_node->key > key)
60          {
61              return _search_helper(current_node->left, key);
62          }
63
64              return _search_helper(current_node->right, key);
65
66      }
67
68      void left_rotate(Node<T>* current_node)
69      {
70          Node<T>* right_child = current_node->right;
71
72          current_node->right = right_child->left;
73          if (right_child->left != nullptr) right_child->left->parent = current_node;
74
75          right_child->left = current_node;
76
77          right_child->parent = current_node->parent;
78
79          if (current_node->parent == nullptr)
80          {
81              this->root = right_child;
82              current_node->parent = right_child;
83
84          }
85          else if (current_node == current_node->parent->left)
86          {
87              current_node->parent->left = right_child;
88              current_node -> parent = right_child;
89          }
90          else if (current_node == current_node->parent->right)
91          {
92              current_node->parent->right = right_child;
93              current_node -> parent = right_child;
94          }
95      }
96
97      void right_rotate(Node<T>* current_node)
98      {
99          Node<T>* left_child = current_node->left;
100
101         current_node->left = left_child->right;
102
103         if (left_child->right != nullptr) left_child->right->parent = current_node;
```

```
104
105         left_child->right = current_node;
106
107         left_child->parent = current_node->parent;
108
109         if (current_node->parent == nullptr)
110         {
111             this->root = left_child;
112             current_node->parent = left_child;
113         }
114         else if (current_node == current_node->parent->left)
115         {
116             current_node->parent->left = left_child;
117             current_node -> parent = left_child;
118         }
119         else if (current_node == current_node->parent->right)
120         {
121             current_node->parent->right = left_child;
122             current_node -> parent = left_child;
123         }
124     }
125
126     void insert(int key, T value, int priority = -1) {
127         Node<T>* val = new Node<T>(key, value, priority);
128         // First, if the tree is empty, just make this the root node
129         if (root == nullptr) {
130             root = val;
131             return;
132         }
133         // Again, I need a helper function to handle my recursion input
134         return _insert_helper(root, val);
135     }
136
137     void _insert_helper(Node<T>* current_node, Node<T>* inserting) {
138         // TODO
139         if (current_node->key >= inserting->key)
140         {
141             if (current_node->left == nullptr)
142             {
143                 current_node->left = inserting;
144                 inserting->parent = current_node;
145             }else
146             {
147                 _insert_helper(current_node->left, inserting);
148             }
149
150             if (current_node->left->priority > current_node->priority)
151             {
152                 right_rotate(current_node);
153             }
154
155         }
156
157
```

```
158         else if (current_node->key < inserting->key)
159         {
160             if (current_node->right == nullptr)
161             {
162                 current_node->right = inserting;
163                 inserting->parent = current_node;
164             }else
165             {
166                 _insert_helper(current_node->right, inserting);
167             }
168 
169             if (current_node->right->priority > current_node->priority)
170             {
171                 left_rotate(current_node);
172             }
173         }
174     }
175 
176     void remove(int key) {
177         return _remove_helper(root, key);
178     }
179 
180     void _remove_helper(Node<T>* current_node, int key) {
181         // TODO
182         if (current_node == nullptr)
183         {
184             return;
185         }
186 
187         if (current_node->key > key)
188         {
189             _remove_helper(current_node->left, key);
190         }
191         else if (current_node->key < key)
192         {
193             _remove_helper(current_node->right, key);
194         }
195         else
196         {
197             current_node->priority = 0;
198 
199             while (! current_node->is_leaf())
200             {
201                 if (current_node->has_left() && current_node->has_right())
202                 {
203                     if (current_node->left->priority >=
                        ↪  current_node->right->priority)
204                     {
205                         right_rotate(current_node);
206                         continue;
207                     }
208 
209                     if (current_node->right->priority >
                        ↪  current_node->left->priority)
```

```
                    {
                        left_rotate(current_node);
                        continue;
                    }
                }

                else if (current_node->has_left()) right_rotate(current_node);

                else if (current_node->has_right()) left_rotate(current_node);
            }

            Node<T>* parent = current_node->parent;
            if (parent == nullptr)
            {
                root = nullptr;
                delete current_node;
            }

            else if (parent->left == current_node)
            {
                delete current_node;
                parent->left = nullptr;
            }

            else
            {
                delete current_node;
                parent->right = nullptr;
            }
        }
    }

    void update(int key, T value) {
        Node<T>* node = search(key);
        if (node) node->value = value;
        else insert(key, value);
    }
};

int ontrack_test() {
    // Ontrack tests, do not change
    BinarySearchTree<string> bst;
    Node<string>* context_node = nullptr;
    string menu = "0. Show this menu\n1. Insert a key\n2. Update a key\n3. Search
    ↪  for a key\n4. Remove a key\n5. Exit";
    int command = 0;
    while (command != 5) {
        if (command == 0) {
            cout << menu << endl;
        } else if (command == 1) {
            //insert
            int key;
            string value;
            int priority;
```

```
263             cout << "Enter key, value, priority: " << endl;
264             cin >> key;
265             cin >> value;
266             cin >> priority;
267             bst.insert(key, value, priority);
268         } else if (command == 2) {
269             //update
270             int key;
271             string value;
272             cout << "Enter key, value: " << endl;
273             cin >> key;
274             cin >> value;
275             bst.update(key, value);
276         } else if (command == 3) {
277             //search
278             cout << "Enter key to search: " << endl;
279             int key;
280             cin >> key;
281             context_node = bst.search(key);
282             if (context_node) {
283                 cout << "Found node (" << context_node->key << ", " <<
                    ↪  context_node->value << ", " << context_node->priority << ")\n";
284                 cout << "Parent: " << (context_node->parent ?
                    ↪  to_string(context_node->parent->key) : "nullptr") << " Left: "
                    ↪  << (context_node->left ? to_string(
285 context_node->left->key) : "nullptr") << " Right: " << (context_node->right ?
    ↪  to_string(context_node->right->key) : "nullptr") << endl;
286             } else cout << "Could not find node with key: " << key << endl;
287         } else if (command == 4) {
288             //remove
289             cout << "Enter key to remove: " << endl;
290             int key;
291             cin >> key;
292             bst.remove(key);
293         }
294         cout << "Enter command (0-5)" << endl;
295         if (!(cin >> command)) {
296             cout << "ERROR: could not read command" << endl;
297             return 1;
298         }
299     }
300     return 0;
301 }
302
303
304 int main(int argc, char** argv) {
305     if (argc == 2) return ontrack_test();
306
307     BinarySearchTree<string> bst;
308     bst.insert(3, "a", 8);
309     bst.insert(11, "b", 6);
310     bst.insert(8, "c", 10);
311     // At this point, c should be the root, and a/b should be left/right
```

```
312     cerr << "Root: (" << bst.root->value << ", " << bst.root->priority << ")" <<
        ↪  endl;
313     cerr << "Left: (" << bst.root->left->value << ", " << bst.root->left->priority
        ↪  << ")" << endl;
314     cerr << "Right: (" << bst.root->right->value << ", " <<
        ↪  bst.root->right->priority << ")" << endl;
315     bst.insert(7, "d", 5);
316     bst.insert(9, "e", 4);
317     bst.insert(5, "f", 2);
318     // Now we should see the final structure of the tree.
319     cerr << "===" << endl;
320     cerr << "Root: (" << bst.root->value << ", " << bst.root->priority << ")" <<
        ↪  endl; // c
321     cerr << "Left: (" << bst.root->left->value << ", " << bst.root->left->priority
        ↪  << ")" << endl; // a
322     cerr << "Right: (" << bst.root->right->value << ", " <<
        ↪  bst.root->right->priority << ")" << endl; // b
323     cerr << "Left->Right: (" << bst.root->left->right->value << ", " <<
        ↪  bst.root->left->right->priority << ")" << endl; // d
324     cerr << "Left->Right->Left: (" << bst.root->left->right->left->value << ", " <<
        ↪  bst.root->left->right->left->priority << ")" << endl; // f
325     cerr << "Right->Left: (" << bst.root->right->left->value << ", " <<
        ↪  bst.root->right->left->priority << ")" << endl; // e
326
327     return 0;
328 }
329
330
331
332 // Tests:
333 /*
334 int main()
335 {
336     BinarySearchTree<string> bst;
337
338     bst.insert(1, "a", 4);
339     bst.insert(5, "b", 9);
340     bst.insert(3, "c", 6);
341     bst.insert(2, "d", 8);
342     bst.insert(4, "e", 11);
343
344     cout << "Root: "
345          << bst.root->key << " "
346          << bst.root->value << " "
347          << bst.root->priority << endl;
348
349     cout << "Root left: "
350          << bst.root->left->key << endl;
351
352     cout << "Root right: "
353          << bst.root->right->key << endl;
354
355     cout << "Root left left: "
356          << bst.root->left->left->key << endl;
```

```
357
358     cout << "Root left right: "
359          << bst.root->left->right->key << endl;
360
361     cout << "\n--- Search Test ---" << endl;
362
363     Node<string>* result1 = bst.search(3);
364
365     if (result1 != nullptr)
366     {
367         cout << "Found key 3: " << result1->value << endl;
368     }
369     else
370     {
371         cout << "Key 3 not found" << endl;
372     }
373
374     Node<string>* result2 = bst.search(100);
375
376     if (result2 != nullptr)
377     {
378         cout << "Found key 100: " << result2->value << endl;
379     }
380     else
381     {
382         cout << "Key 100 not found" << endl;
383     }
384
385     cout << "\n--- Remove Non-existing Key Test ---" << endl;
386
387     bst.remove(100);
388
389     cout << "Root after remove(100): " << bst.root->key << endl;
390     cout << "Search 100: "
391          << (bst.search(100) == nullptr ? "Not found" : "Found")
392          << endl;
393
394
395     cout << "\n--- Remove Leaf Test ---" << endl;
396
397     bst.remove(1);
398
399     cout << "Search 1: "
400          << (bst.search(1) == nullptr ? "Not found" : "Found")
401          << endl;
402
403     cout << "Root: " << bst.root->key << endl;
404     cout << "Root left: " << bst.root->left->key << endl;
405     cout << "Root right: " << bst.root->right->key << endl;
406     cout << "Root left right: " << bst.root->left->right->key << endl;
407
408     cout << "\n--- Remove One-child Test ---" << endl;
409
410     bst.remove(2);
```

```
411
412     cout << "Search 2: "
413          << (bst.search(2) == nullptr ? "Not found" : "Found")
414          << endl;
415
416     cout << "Search 3: "
417          << (bst.search(3) == nullptr ? "Not found" : "Found")
418          << endl;
419
420     cout << "Root: " << bst.root->key << endl;
421     cout << "Root left: " << bst.root->left->key << endl;
422     cout << "Root right: " << bst.root->right->key << endl;
423
424     cout << "Root left parent: "
425          << bst.root->left->parent->key << endl;
426
427     cout << "\n--- Remove Two-children Root Test ---" << endl;
428
429     BinarySearchTree<string> bst2;
430
431     bst2.insert(1, "a", 4);
432     bst2.insert(5, "b", 9);
433     bst2.insert(3, "c", 6);
434     bst2.insert(2, "d", 8);
435     bst2.insert(4, "e", 11);
436
437     bst2.remove(4);
438
439     cout << "Search 4: "
440          << (bst2.search(4) == nullptr ? "Not found" : "Found")
441          << endl;
442
443     cout << "Root: " << bst2.root->key << endl;
444     cout << "Root left: " << bst2.root->left->key << endl;
445     cout << "Root left left: "
446          << bst2.root->left->left->key << endl;
447     cout << "Root left right: "
448          << bst2.root->left->right->key << endl;
449
450     cout << "Root parent is nullptr: "
451          << (bst2.root->parent == nullptr ? "Yes" : "No")
452          << endl;
453
454     cout << "Root left parent: "
455          << bst2.root->left->parent->key << endl;
456
457     cout << "\n--- Remove Single Root Test ---" << endl;
458
459     BinarySearchTree<string> bst3;
460     bst3.insert(10, "only", 100);
461
462     bst3.remove(10);
463
464     cout << "Root is nullptr: "
```

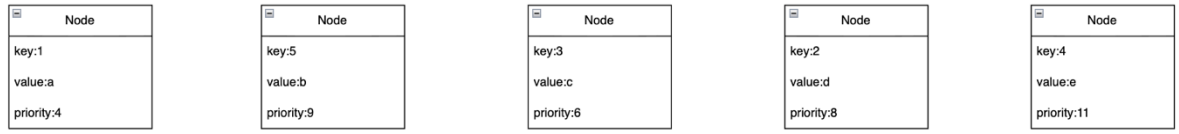
```
465         << (bst3.root == nullptr ? "Yes" : "No")
466         << endl;
467
468     cout << "Search 10: "
469         << (bst3.search(10) == nullptr ? "Not found" : "Found")
470         << endl;
471
472     cout << "\n--- Remove All Test ---" << endl;
473
474     BinarySearchTree<string> bst4;
475
476     bst4.insert(1, "a", 4);
477     bst4.insert(5, "b", 9);
478     bst4.insert(3, "c", 6);
479     bst4.insert(2, "d", 8);
480     bst4.insert(4, "e", 11);
481
482     bst4.remove(1);
483     bst4.remove(2);
484     bst4.remove(3);
485     bst4.remove(4);
486     bst4.remove(5);
487
488     cout << "Tree is empty: "
489         << (bst4.root == nullptr ? "Yes" : "No")
490         << endl;
491
492     cout << "Search 1: "
493         << (bst4.search(1) == nullptr ? "Not found" : "Found")
494         << endl;
495
496     cout << "Search 5: "
497         << (bst4.search(5) == nullptr ? "Not found" : "Found")
498         << endl;
499
500     cout << "\n--- Remove Single-left-child Test ---" << endl;
501
502     BinarySearchTree<string> bst5;
503
504     bst5.insert(10, "a", 10);
505     bst5.insert(5, "b", 5);
506
507     bst5.remove(10);
508
509     cout << "Search 10: "
510         << (bst5.search(10) == nullptr ? "Not found" : "Found")
511         << endl;
512
513     cout << "Search 5: "
514         << (bst5.search(5) == nullptr ? "Not found" : "Found")
515         << endl;
516
517     cout << "Root: " << bst5.root->key << endl;
518
```

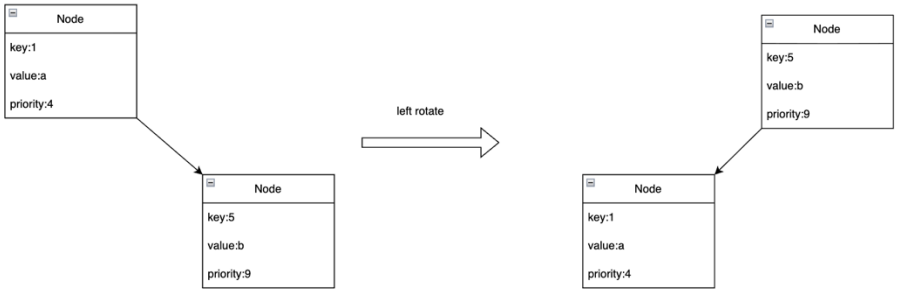
```
519     cout << "Root parent is nullptr: "
520          << (bst5.root->parent == nullptr ? "Yes" : "No")
521          << endl;
522 
523     return 0;
524 }
525 
526 */
```

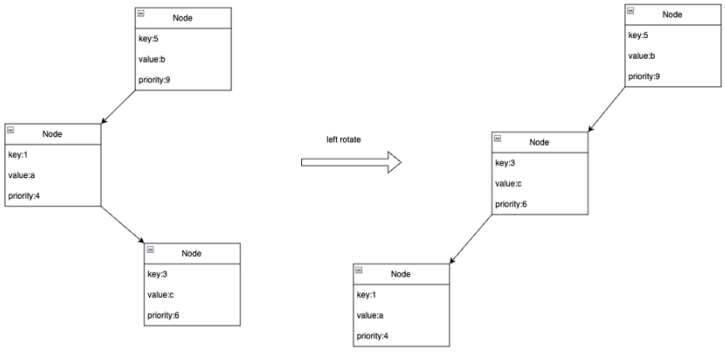
HD2 Report

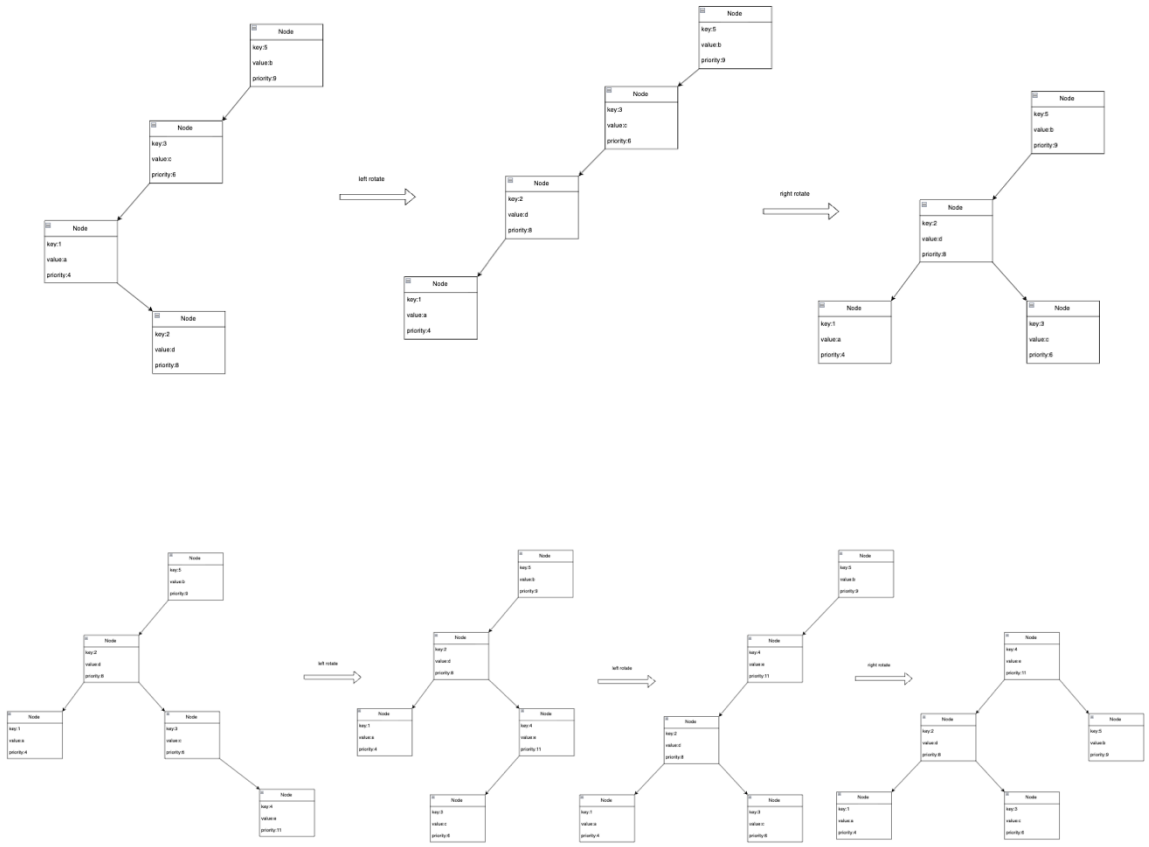
Worked Example

Each node in the Binary Search Heap (BSH) must satisfy two properties: the keys follow the Binary Search Tree ordering, while the priorities follow a max-heap ordering, where a parent's priority is at least as large as its children's priorities. Each new node is first inserted according to its key and is then rotated upwards while its priority violates the heap property.









Reasoning

A standard Binary Search Tree can become very tall when keys are inserted in an unfavourable order. For example, inserting already sorted keys can create a chain with height n, causing retrieval, insertion and removal to degrade to O(n).

The BSH reduces the influence of insertion order by assigning each node a random priority. After normal BST insertion, rotations move a node with a higher priority above nodes with lower priorities. Therefore, the final structure depends not only on the order of the keys but also on the independently generated priorities.

For a very tall BSH to form, the randomly generated priorities would need to occur in a particular relative order that is compatible with the long chain of keys. As more nodes are added, obtaining such an extreme ordering becomes increasingly unlikely. Instead, high-priority nodes tend to be distributed throughout the key range and are rotated towards the top of the tree, dividing the remaining keys into smaller subtrees.

Consequently, although a worst-case tree of height O(n) is still possible, it is unlikely with random priorities. The expected height is O(log n), so operations with complexity O(h) are expected to perform efficiently in most cases.

Evidence of Testing

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/HD2/main
Root: 4 e 11
Root left: 2
Root right: 5
Root left left: 1
Root left right: 3

--- Search Test ---
Found key 3: c
Key 100 not found

--- Remove Non-existing Key Test ---
Root after remove(100): 4
Search 100: Not found

--- Remove Leaf Test ---
Search 1: Not found
Root: 4
Root left: 2
Root right: 5
Root left right: 3

--- Remove One-child Test ---
Search 2: Not found
Search 3: Found
Root: 4
Root left: 3
Root right: 5
Root left parent: 4
```

```
--- Remove Two-children Root Test ---
Search 4: Not found
Root: 5
Root left: 2
Root left left: 1
Root left right: 3
Root parent is nullptr: Yes
Root left parent: 5

--- Remove Single Root Test ---
Root is nullptr: Yes
Search 10: Not found

--- Remove All Test ---
Tree is empty: Yes
Search 1: Not found
Search 5: Not found

--- Remove Single-left-child Test ---
Search 10: Not found
Search 5: Found
Root: 5
Root parent is nullptr: Yes
```

20 Applied Engagement

Engagement with Applied activities with your Buddy Group

Outcome	Description
TLO1	Practice effective collaboration through working with your group on the provided case studies.

Date	Author	Comment
2026/08/11 15:11	Yihao Yu	Working On It
2026/09/11 00:50	Yihao Yu	Hi Gary, thanks for your reply in advance. For the P10, I just want to know what is the meaning of "The only thing that matters for this task is having engaged with the Applied activity and your Buddy Group. You won't be marked on the key learnings you shared. " and "the marking of it is only based on having done the activity". I mean if that means once I attend the applied and finish the applied tasks, then I could pass the P10? Meanwhile, I also want to double check that if I need to provide more details for what I have done for the applied tasks in my report? Such as screenshot, the draft of the tasks and so on. Thanks again for your patience and hard work!
2026/09/11 00:50	Yihao Yu	Ready for Feedback
2026/09/11 14:02	Gary Zheng	Hi Yihao, please see me next class and we can talk about this.
2026/09/11 14:02	Gary Zheng	**Automated Message:** Task "P10 Applied Engagement" will be graded during portfolio assessment only. You can keep submitting it for feedback before the task deadline, but you must still submit it directly for portfolio assessment before the portfolio deadline.
2026/09/11 14:02	Gary Zheng	Working On It
2026/09/29 17:09	Yihao Yu	Ready for Feedback
2026/09/29 18:31	Gary Zheng	Complete

Monash

Fundamentals of Algorithms

OnTrack Submission

Applied Engagement

Submitted By:
Yihao Yu
yyuu0140
2026/09/29 17:09

Tutor:
Gary Zheng

Task Outcomes		Supports
TLO1	Practice effective collaboration through working with your group on the provided case studies.	

September 29, 2026

Applied and Buddy Group Engagement Report

First Activity

- Week number: 2
- Applied class date and time: 04/08/2026 2:00pm – 4:00pm
- TA name: Gary Zheng

Key Learning 1
I learned how different sorting algorithms, including Bubble Sort, Insertion Sort, and Selection Sort, organise data using different approaches. Working through these algorithms with my Buddy Group helped me better understand how elements are compared, swapped, or shifted during the sorting process.

Key Learning 2
I learned how sorting algorithms can be applied to a practical problem through the Sort and Store case study. Our group designed a solution that temporarily stored incoming users and sorted them by timestamp before sending them to the database. We used an insertion-sort-style approach to order the timestamps, which helped me understand how a sorting algorithm can be adapted and used as part of a larger system rather than only for sorting a simple array.

Second Activity

- Week number: 9
- Applied class date and time: 29/09/2026 2:00pm – 4:00pm
- TA name: Gary Zheng

Key Learning 1

I learned how a Max Heap can be used to efficiently manage tasks based on priority. In the activity, we used heap insertion for adding a single task in O(log n), and bottom-up heapify for adding a batch of tasks in O(n+m). This helped me understand why repeatedly inserting every task in a batch would be less efficient, and how choosing the appropriate heap operation can satisfy different time-complexity requirements.

Key Learning 2

I learned that a heap is efficient for accessing the highest-priority task, but becomes more challenging when an additional constraint is introduced. In the Task Manager, the highest-priority task may not fit within minutes_left, so we recursively searched the heap for the best task that could be completed in the available time. After finding it, we removed it and restored the heap property using sink or bubble_up. This helped me understand both the advantages and limitations of heaps when solving problems with multiple constraints.

21 Problem Solving Workbook

Solve complex problems, learn what they need, design similar problems, and present them to us.

Outcome	Description
TLO1	Engage in independent research and problem solving, and practice effective presentation of the new problems and solutions you find.

Date	Author	Comment
2026/10/03 04:21	Yihao Yu	Working On It
2026/10/13 00:07	Yihao Yu	Ready for Feedback
2026/10/13 00:11	Yihao Yu	Ready for Feedback

Monash

Fundamentals of Algorithms

OnTrack Submission

Problem Solving Workbook

Submitted By:
Yihao Yu
yyuu0140
2026/10/13 00:07

Tutor:
Ali Toosi

Task Outcomes		Supports
TLO1	Engage in independent research and problem solving, and practice effective presentation of the new problems and solutions you find.	

October 13, 2026



Zip submission file

This zip file was uploaded with the submission. Zip contents are not expanded into this PDF.

Download submitted files (Must be already authenticated in OnTrack)

Zip contents

```
↪ 4K/
  ↪ Faculty_Check-in/
    ↪ .DS_Store
    ↪ Faculty Check.pdf
    ↪ stackqueue2.cpp
    ↪ video1894559910.mp4
  ↪ Perfect_Test/
    ↪ .DS_Store
    ↪ heap1.cpp
    ↪ Perfect Test Report.pdf
    ↪ video1643908467.mp4
  ↪ .DS_Store
↪ __MACOSX/
  ↪ 4K/
    ↪ Faculty_Check-in/
      ↪ ._.DS_Store
      ↪ ._Faculty Check.pdf
      ↪ ._stackqueue2.cpp
      ↪ ._video1894559910.mp4
    ↪ Perfect_Test/
      ↪ ._.DS_Store
      ↪ ._heap1.cpp
      ↪ ._Perfect Test Report.pdf
      ↪ ._video1643908467.mp4
    ↪ ._.DS_Store
  ↪ ._4K
```

22 HD Interview

This task is to keep track of your HD interview. Once you've finished all your HD tasks, and have passed your Competency Hurdle test, we'll email you with interview details.

Date	Author	Comment
2026/10/06 14:49	Yihao Yu	Working On It
2026/10/13 13:12	Yihao Yu	Ready for Feedback